Trajectory planning for robot manipulators and grasping

Aljaz Kramberger

alk@mmmi.sdu.dk

SDU Robotics

The Maersk Mc-Kinney Moller Institute

University of Southern Denmark

Aljaz Kramberger

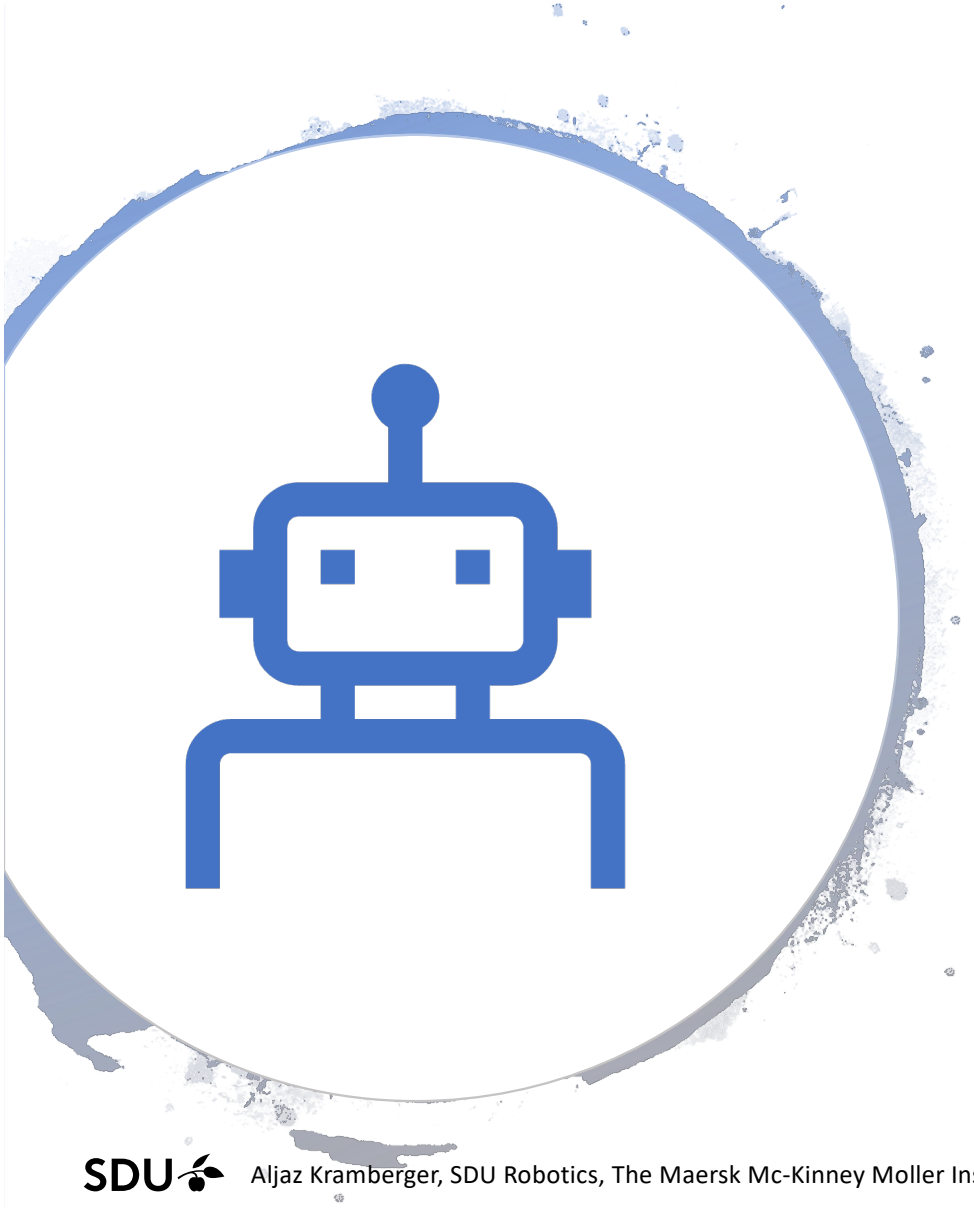
Associate Professor

Email: alk@mmmi.sdu.dk
Office: Ø27-602a-3
SDU Robotics, The Maersk Mc-Kinney Moller Institute.

M.Sc – Mechatronics,
Ph.D – Robotics (robot control in contact with the environment)

Research filed:

- → Robot control, learning, human robot interaction, industrial robotics,
- → Part design and optimization,
- → Robot grasping



Agenda

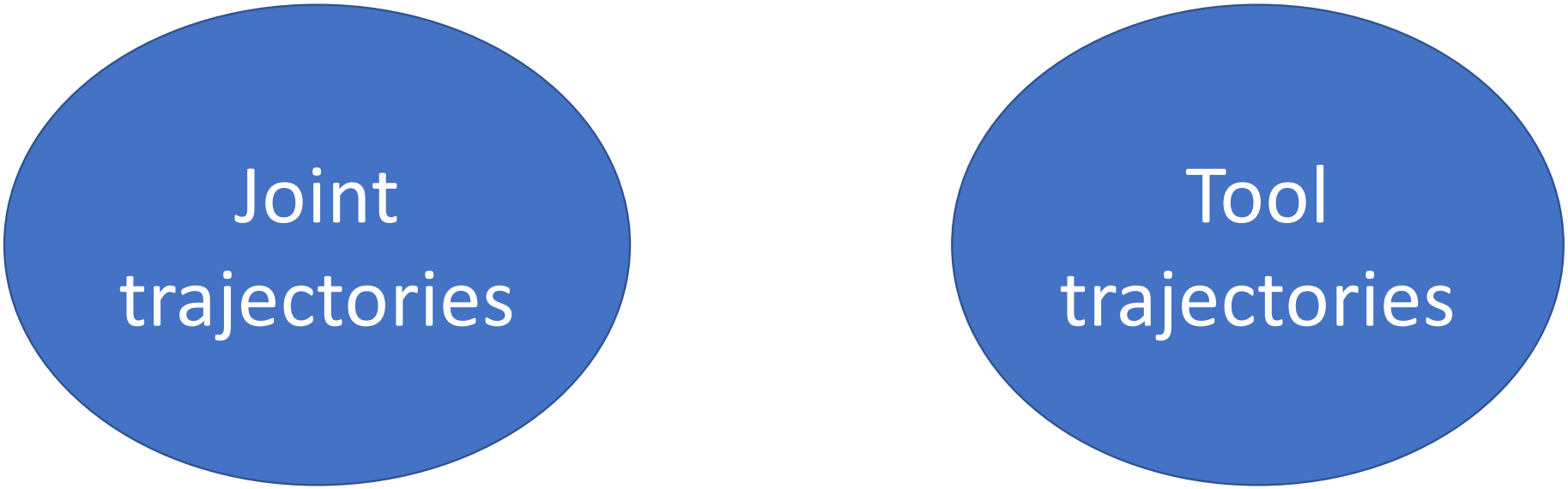
- Interpolation for trajectories represented in Cartesian space
- Cubic polynomial interpolation
- Defining trajectories by Learning
- Robotic grasping

Additional material

- Siciliano, Bruno, Oussama Khatib, and Torsten Kröger, eds. *Springer handbook of robotics*. Vol. 200. Berlin: springer, 2008.
- Siciliano, Bruno, Lorenzo Sciavicco, Luigi Villani, and Giuseppe Oriolo. *Robotics: modelling, planning and control*. Springer Science & Business Media, 2010.
- Mihelj, Matjaž, Tadej Bajd, Aleš Ude, Jadran Lenarčič, Aleš Stanovnik, Marko Munih, Jure Rejc, and Sebastjan Šlajpah. *Robotics*. Springer, Cham, 2019.

Trajectory's

General we distinguish…



Differences

- Joint trajectories are presented as:
  - $Trj_{joint} = \{q_i^1, q_i^2, \dots q_i^n, t_i\}_{i=1}^{T,n}$, t_i , $i = 1, \dots, T_i$ are the number of samples and n number of joints.
  - can directly be send to the robot.
  - precise deterministic execution – (better tracking),
  - the movement is not necessary linear although it was constructed such.

- Tool trajectories are presented as:
  - $Trj_{tool} = \{p_i, o_i, t_i\}_{i=1}^{T}$ where $p = (p_x, p_y, p_z)$ is the positional part and $o = \boldsymbol{R}$ orientational part of the trajectory.
  - Invers kinematics is needed in order to execute the trajectory.

Trajectory construction

- Trajectories can be constructed in different ways:
  - trajectories can be acquired with demonstration,
  - simple construction with point to point interpolation,
  - splines and polynomials,
  - using dynamic systems,
  - etc.

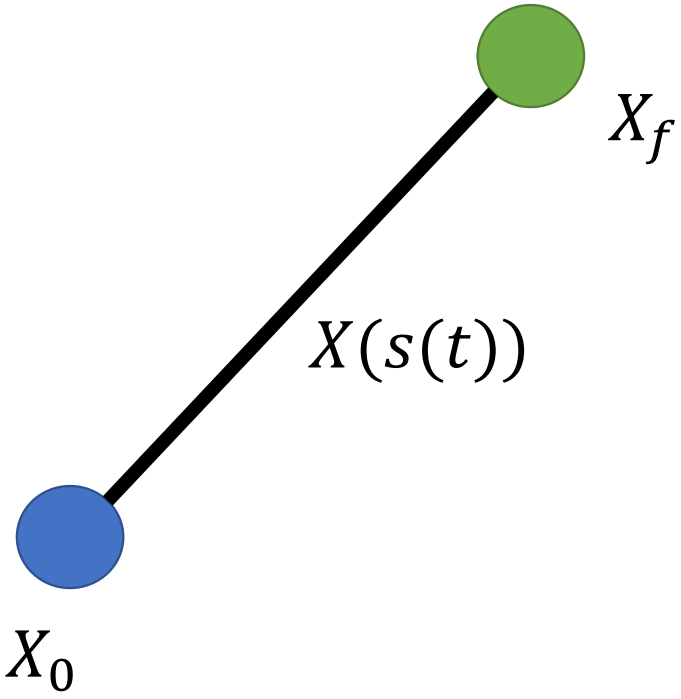
Linear interpolation

Linear interpolation

- Let's consider two vectors X_0 and X_f. A linear interpolation between them is given as:

  $X(s(t)) = X_0 + (X_f - X_0)s(t) \quad 0 \leq s \leq 1$

- where $s(t)$ denotes a non decreasing continuous time profile satisfying:
  - $s(t_0) = 0$ at start,
  - $s(t_f) = 1$ at the end of the movement.



Linear interpolation

- We can also define a linear interpolator in joint space:

$$q^i(t) = q_o^i + \frac{t - t_0}{t_f - t_0}\left(q_f^i - q_o^i\right), i = 1, \ldots, \text{numDOF}$$

- It is given by a first order polynomial, for better results higher order polynomials work better:
  - the problem of velocity – discontinuity and unbounded acceleration in both ends.
- the outcome, compared with tool space interpolation is not equal!!!!!
- therefore, we have to carefully consider which type of interpolation is appropriate for the desired task/application.

Linear interpolation

- For velocity we will consider the following:

$$s(t) = \frac{t - t_0}{t_f - t_0} \ (constant\ velocity)$$

$$s(t) = \left(\frac{t - t_0}{t_f - t_0}\right)^2 \ (linear\ ramp - up\ of\ velocity)$$

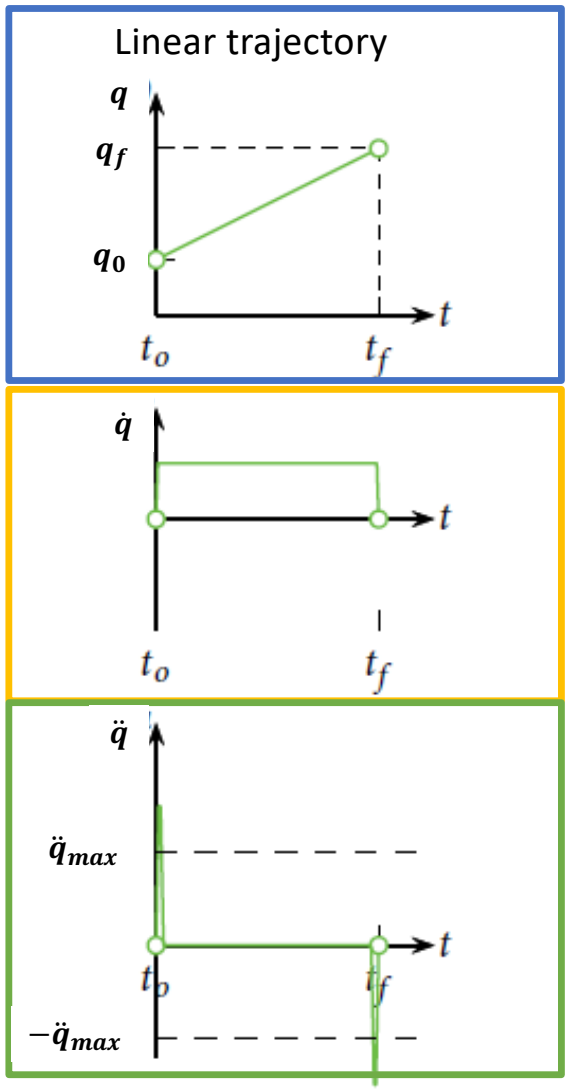
$$s(t) = 1 - \left(\frac{t - t_0}{t_f - t_0}\right)^2 \ (linear\ ramp - down\ of\ velocity)$$

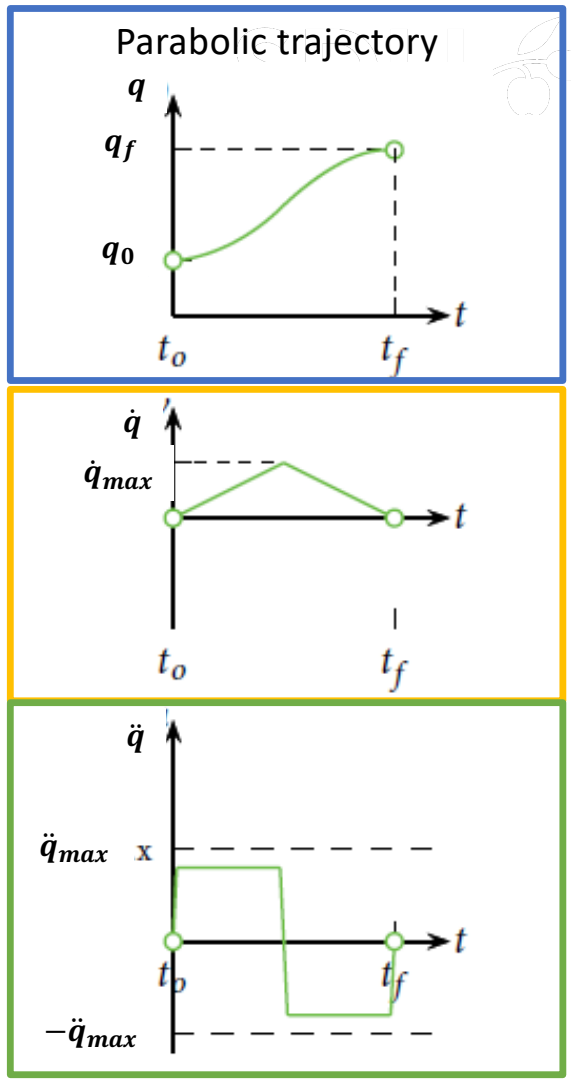
Trajectories

POSITION

VELOCITY

ACCELERATION



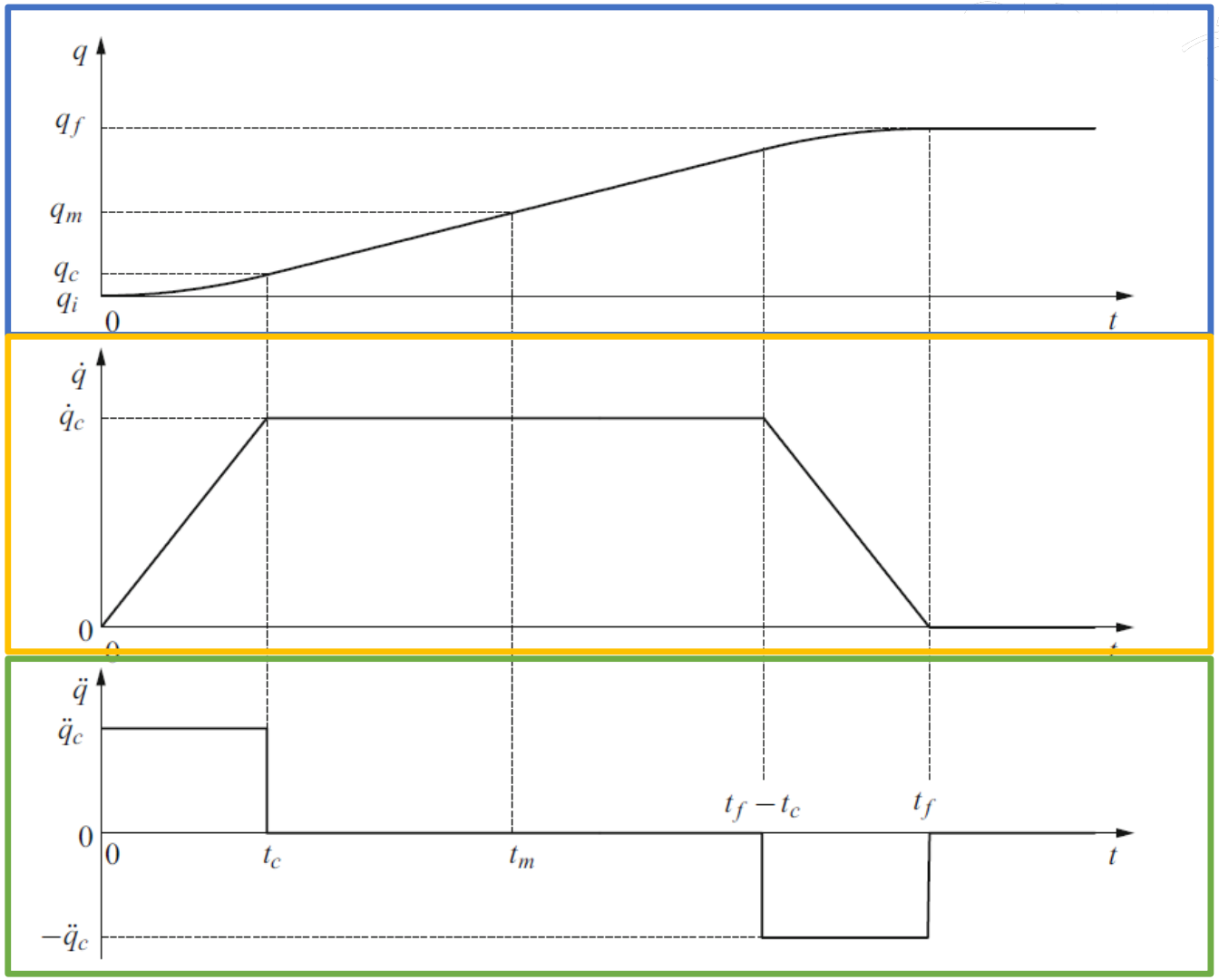


Linear interpolation

POSITION

VELOCITY

ACCELERATION



Parabolic blend in Task space

- Parabolic blend in task space, can for the positional part be calculated in the same way as for joint space parabolic blends.
- Orientation can be computed with piece vise computing slerp trajectories in quaternion space (more info *Dantam N, Stilman M. Spherical parabolic blends for robot workspace trajectories. In2014 IEEE/RSJ International Conference on Intelligent Robots and Systems 2014 Sep 14 (pp. 3624-3629)*).
- Followed by solving the inverse kinematics for the generated task space path.

Quaternions

- Quaternions allow stable and constant interpolation of orientations
- Compact(-ish) representation
- Also easy to concatenate
- Faster multiplication algorithms to combine successive rotations than using rotation matrices
- Easier to normalize than rotation matrices
- Mathematically stable – suitable for statistics
- Given an angle and axis, easy to convert to and from quaternion

Definition

- We can extend complex numbers to 3-dimensional space by adding two imaginary numbers to our number system in addition to $\boldsymbol{i}$
- Quaternions have 4 dimensions (each quaternion consists of 4 scalar numbers), one real dimension and 3 imaginary dimensions.
- Each of these imaginary dimensions has a unit value of the square root of -1, but they are different square roots of -1 all mutually perpendicular to each other, known as $\boldsymbol{i}$, $\boldsymbol{j}$ and $\boldsymbol{k}$.
- We can represent 3D rotations as 3 numbers (e.g. Euler angles) but such a representation is non-linear and difficult to work with.
- An analogy of a two dimensional map of the earth

Definition

- A quaternion can be represented as follows:

$$q = w + x\boldsymbol{i} + y\boldsymbol{j} + z\boldsymbol{k} \quad s, x, y, z \in \mathbb{R}$$

- Various annotation can be used:

$$\begin{aligned} q &= [w, \boldsymbol{v}] \\ &= [w, x\boldsymbol{i} + y\boldsymbol{j} + z\boldsymbol{k}] \\ &= w + x\boldsymbol{i} + y\boldsymbol{j} + z\boldsymbol{k} \end{aligned}$$

- In literature scalar and vector part are sometimes flipped $q = [\boldsymbol{v}, w]$

Quaternion interpolation

Linear interpolation

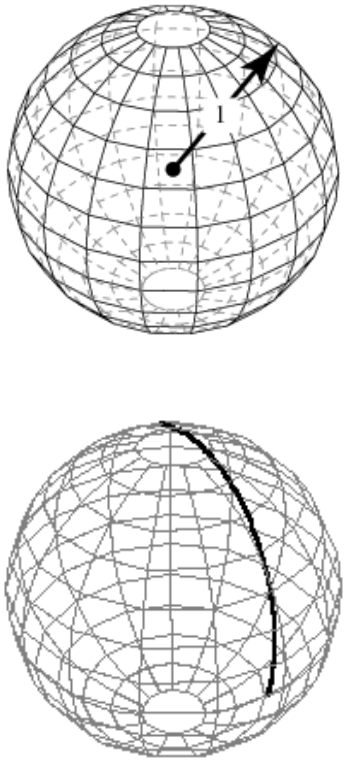
- Interpolation is used to determine intermediate points between two given points $\boldsymbol{Q}_1$ and $\boldsymbol{Q}_2$
- Linear interpolation can be used to do interpolation in normal space
- The standard linear interpolation formula is

$$\boldsymbol{Q}^i = \boldsymbol{Q}_1 + t(\boldsymbol{Q_2} - \boldsymbol{Q_1})$$

- The general steps to apply this equation are:
  - Compute the difference between $\boldsymbol{Q}_1$ and $\boldsymbol{Q}_2$
  - Take the fractional part of that difference
  - Adjust the original value by the fractional difference between the two points

Unit sphere

- A quaternion is a point on the 4-D unit sphere
- If we want to interpolate between two points on a sphere, we do not just want to Lerp between them
- Instead, we will travel across the surface of the sphere by following a great arc
- Move with constant angular velocity along the great circle between the two points
- There are two SLERP choices on a sphere – we pick the shortest

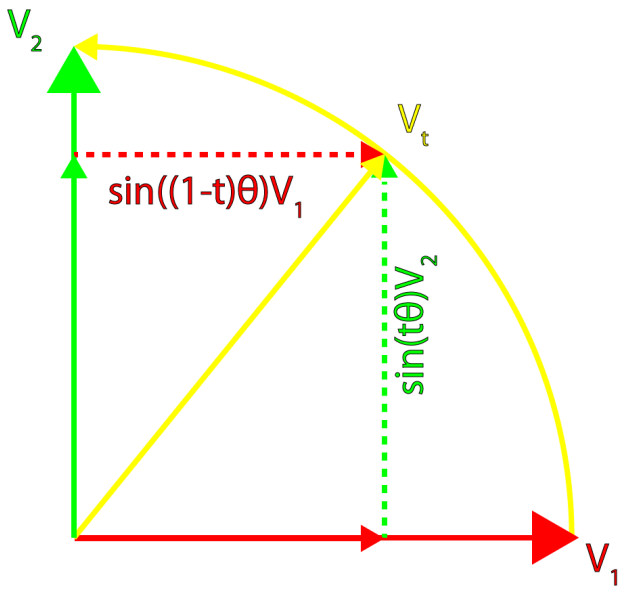


Source: Wolfram Research

Spherical interpolation of vectors

- We will apply a spherical interpolation of vectors to quaternions
- The general form of a spherical interpolation for vectors is defined as:

$$\boldsymbol{v}_t = \frac{\sin(1-t)\theta}{\sin\theta}\boldsymbol{v}_2 + \frac{\sin t\theta}{\sin\theta}\boldsymbol{v}_2$$



Spherical interpolation of quaternions

- The formula for spherical interpolation of vectors can be applied unmodified to quaternions:

$$Q_t = \frac{\sin(1-t)\theta}{\sin\theta} Q_2 + \frac{\sin t\theta}{\sin\theta} Q$$

- Angle θ need to be calculated

Spherical interpolation - angle θ

- We can obtain the angle θ by computing the dot-product between Q_1 and Q_2

$$\cos\theta = \frac{Q_1 \cdot Q_2}{|Q_1||Q_2|} = \frac{w_1 w_2 + x_1 x_2 + y_1 y_2 + z_1 z_2}{|Q_1||Q_2|}$$

$$\theta = \cos^{-1}\left(\frac{w_1 w_2 + x_1 x_2 + y_1 y_2 + z_1 z_2}{|Q_1||Q_2|}\right)$$

Spherical interpolation - considerations

- The quaternion dot-product can results in a negative value
  -> the resulting interpolation will travel the long-way around the 4D sphere
- If the result of the dot product is negative, then we can negate one of the orientations
- Negating the scalar and the vector part of the quaternion does not change the orientation that it represents

Spherical interpolation - considerations

- If the angular difference θ between Q_1 and Q_2 is very small then $\sin\theta$ becomes 0 -> undefined result when we divide by $\sin\theta$.
- In this case, we can fall-back to using linear interpolation between Q_1 and Q_2.
- As changes are small a straight line is close to the arc on the sphere

Quaternion difference

- quaternion logarithm which maps $S^3 \rightarrow \mathbb{R}^3$ is defined as:

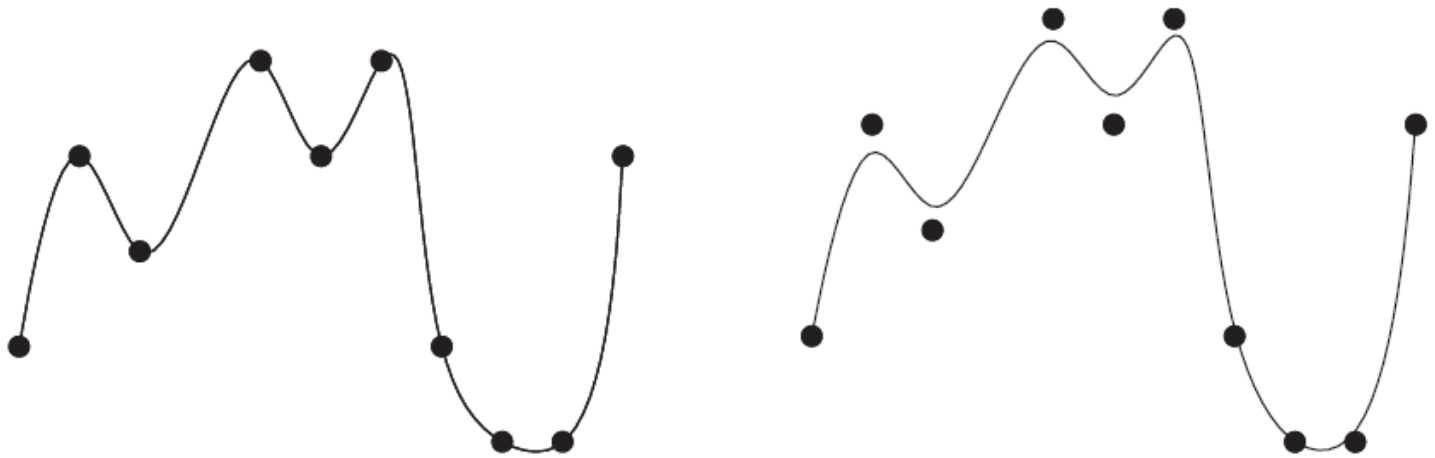
$$\mathbf{r} = \log(\mathrm{Q}) = \log(v + \boldsymbol{u}) = \begin{cases} \arccos(v)\frac{\boldsymbol{u}}{\|\boldsymbol{u}\|}, \boldsymbol{u} \neq 0 \\ [0,0,0]^{\mathrm{T}}, \text{ otherwise} \end{cases}$$

- which can be interpreted as a difference vector $\log(\boldsymbol{Q}_2 * \overline{\boldsymbol{Q}_1})$ between 2 unit quaternions.
- its inverse, the exponential map, which maps from $\mathbb{R}^3 \rightarrow S^3$ is defined:

$$\exp(\boldsymbol{r}) = \begin{cases} \cos(\| \boldsymbol{r} \|) + \sin(\| \boldsymbol{r} \|)\frac{\boldsymbol{r}}{\|\boldsymbol{r}\|}, \boldsymbol{r} \neq 0 \\ 0, \text{ otherwise} \end{cases}$$

From Points, via- points to trajectories

From Points, via- points to trajectories



Interpolation

Approximation

- *Definition: Interpolation* Constructing new data points within the range of a discrete set of known data points (exact fitting).
- *Definition: Approximation* Inexact fitting of a discrete set of known data points.

Interpolation between points

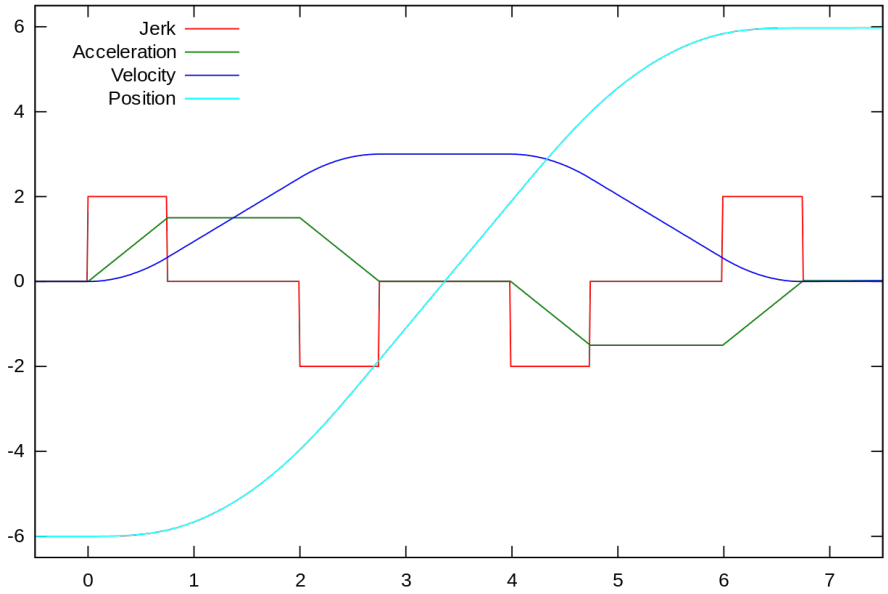
- In comparison, velocities and accelerations are better approximated with the parabolic blends.
- Still this method does not give a completely smooth trajectory!
- For this we must look at the derivative of the acceleration at blend points.

Definition of jerk for motion planning

- Jerk in motion control is the rate at which an object's acceleration changes with respect to time.
- It is a vector quantity (having both magnitude and direction). and expressed in m/s^3(SI units).
- Jerk can be calculated as:

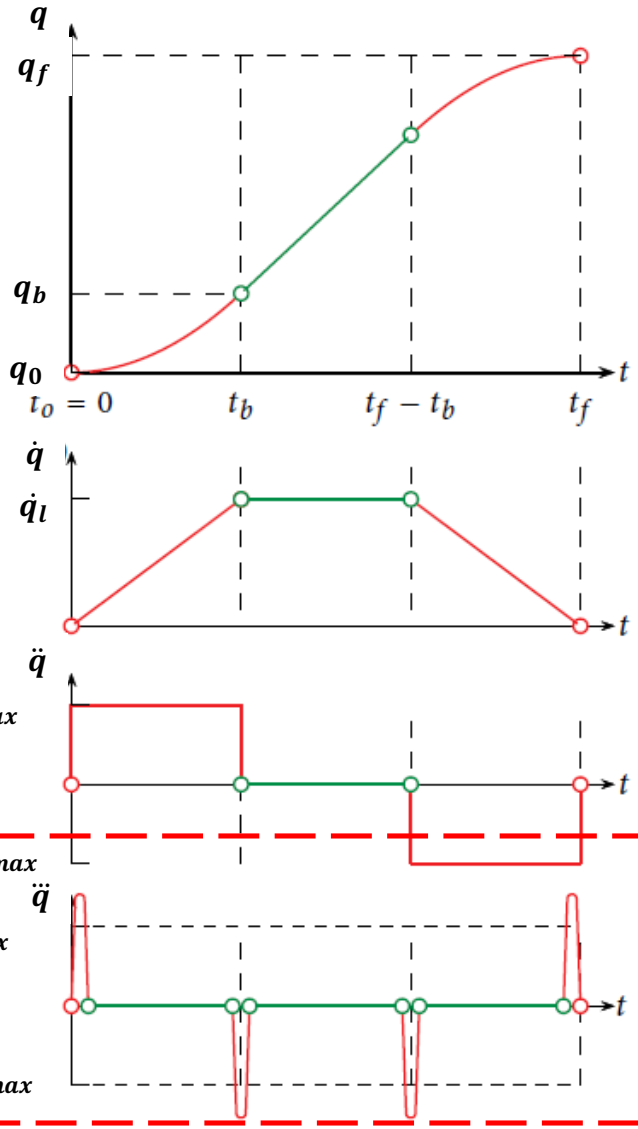
$$\vec{j} = \frac{da(t)}{dt} = \frac{d^2v(t)}{dt} = \frac{d^3q(t)}{dt}$$

- jerk can be calculated for joint and task space trajectories.

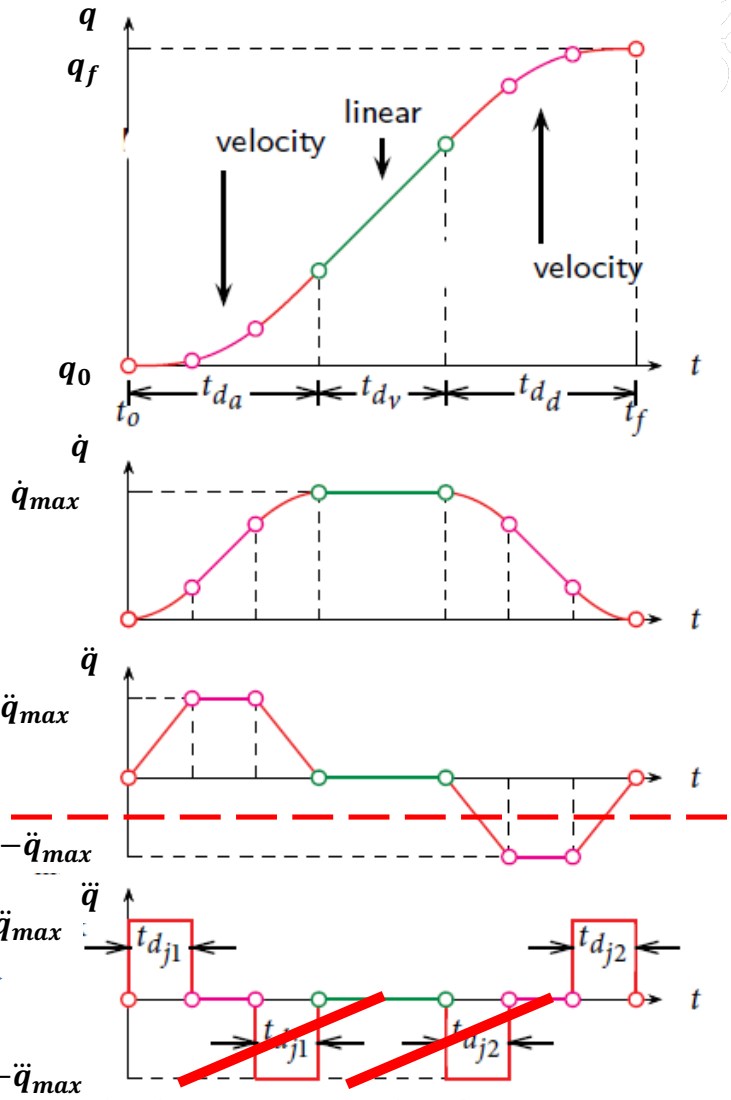


https://commons.wikimedia.org/w/index.php?curid=46643265

How does this show?



Introduction of trapezoidal acc. bled = S blends [L. Biagiotti and C. Melchiorri. *Trajectory Planning for Automatic Machines and Robots*. Springer Verlag, 2008]



Existence of jerk impulse

We are looking for = polynomial interpolation

Cubic Polynomial Interpolation

Cubic Polynomial Interpolation

- Cubic polynomial interpolation can be applied in task and joint space.
- Dense here means that we may assume that the displacements between adjacent frames are small.
- In such a case, linear interpolation is generally not suitable because the segments are two small (velocity is not continues between segments).
- Here, we recommend to use cubic interpolations or splines.

Cubic Polynomial Interpolation

- Cubic interpolation can be used in:
  - joint space, using n-tuple $X_i \equiv q^i$,
  - tool space, using 7-tuple $X_i \equiv (p^i, Q^i)$ for $i = 0,1, \dots, N$.
- Orientations should be represented as quaternions (don't forget to normalize at the end!!).
- Between two frames (X_{i-1} and X_i) we obtain a curve, defined as:

$$X_{i-1,i}(t) = a_i t^3 + b_i t^2 c_i t + d_i$$

- where parameters a_i, b_i, c_i, d_i have to be determined.
- depending on the number of curves we have 4N parameters to determine.

Cubic Polynomial Interpolation

- The curves must satisfy the following constrains

$$X_{i-1,i}(t_{i-1}) = a_i t_{i-1}^3 + b_i t_{i-1}^2 + c_i t_{i-1} + d_i = X_{i-1}\ i = 1, \dots, N$$
$$X_i(t_i) = a_i t_i^3 + b_i t_i^2 + c_i t_i + d_i = X_i\ i = 1, \dots, N$$

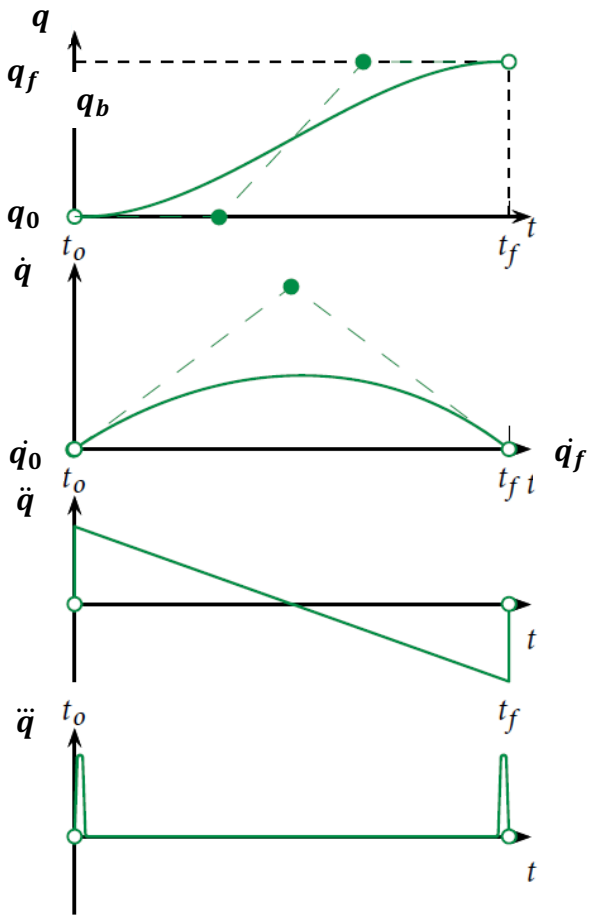
- which gives a 2N linear equations with unknowns,
- the remaining 2N equations can be given in several ways.
- for example if velocities at passage points are given we get

Cubic Polynomial Interpolation

$$\dot{X}_{i-1}(t_{i-1}) = 3a_i t_{i-1}^2 + 2b_i t_{i-1} + c_i = \dot{X}_{i-1} \quad i = 1, \dots, N$$
$$\dot{X}_i(t_i) = 3a_i t_i^2 + 2b_i t_i + c_i = \dot{X}_i \qquad i = 1, \dots, N$$

- which gives 4 linear equations with 4 unknowns for each sub-path.

Let' take a closer look at this problem…



- Let's consider a cub. interp. between two positions in joint space.
- where 4 parameters a0, a1 , a2 , a3 are to be determined by boundary conditions.
- Property: bounded acceleration, jerk impulse at both ends.

$$q(t) = a_0 + a_1(t - t_0) + a_2(t - t_0)^2 + a_3(t - t_0)^3$$
$$t \in [t_0, t_f], t_d \triangleq t_f - t_0$$

$$\begin{cases} q(t_0) = a_0 = q_0 \\ \dot{q}(t_0) = a_1 = \dot{q}_0 \\ q(t_f) = a_0 + a_1 t_d + a_2 t_d^2 + a_2 t_d^3 = q_f \\ \dot{q}(t_f) = a_1 + 2a_2 t_d + 3a_3 t_d^2 = \dot{q}_f \end{cases} \Rightarrow \begin{cases} a_0 = q_0 \\ a_1 = \dot{q}_0 \\ a_2 = \dfrac{3h - (2\dot{q}_0 + \dot{q}_f)t_d}{t_d^2} \\ a_3 = \dfrac{-2h + (\dot{q}_0 + \dot{q}_f)t_d}{t_d^3} \end{cases}$$

Multi point cubic interpolation

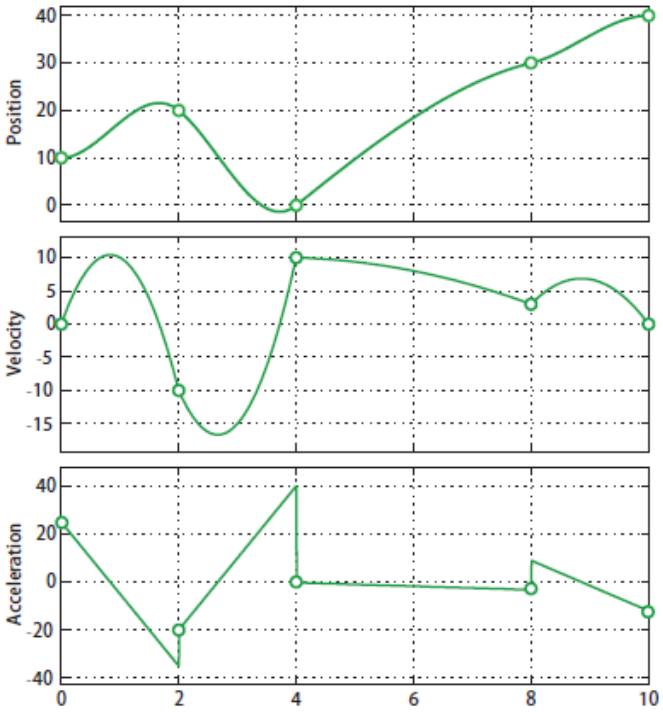
- Given $q_0, q_f, \dot{q}_0, \dot{q}_f$ at t_0, t_f and via points $[q_k]_1^m$ and $[t_k]_1^m$ where m is the number of sections between the points and k is the number of the specific point, we solve:

  $$a_{0k} + a_{1k}(t - t_k) + a_{2k}(t - t_k)^2 + a_{3k}(t - t_k)^3$$

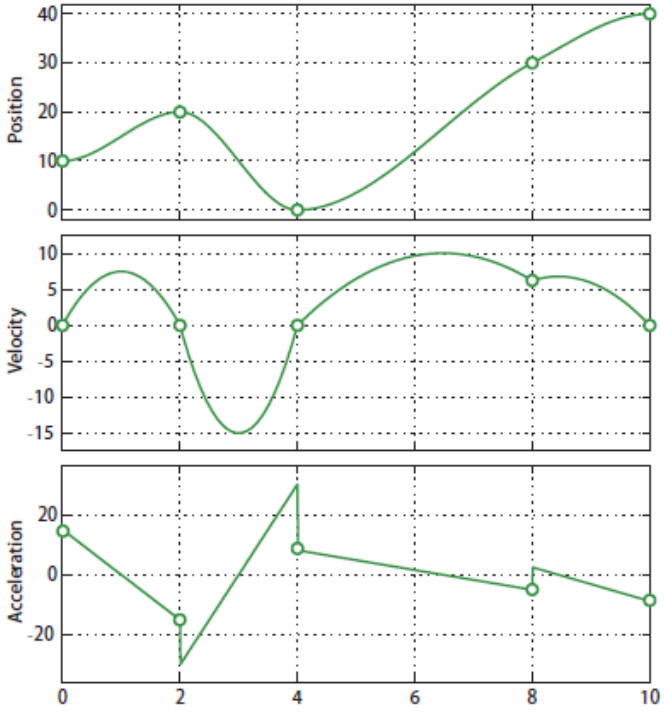
  for all unknowns $\{a_{0k}, a_{1k}, a_{2k}, a_{3k}\}_0^m$.
- accelerations at via points can be assigned by the user $[\dot{q}_k]_1^m$ and $t_d = t_f - t_0, h = q_f - q_0$, we can compute the coefficients.

$$\begin{cases} a_{0k} = q_0 & a_{1k} = \dot{q}_0 \\ a_{2k} = \dfrac{3h - (2\dot{q}_0 + \dot{q}_f)t_d}{t_d^2} & a_{3k} = \dfrac{-2h + (2\dot{q}_0 + \dot{q}_f)t_d}{t_d^3} \end{cases}, k = 0,1, \dots, m$$

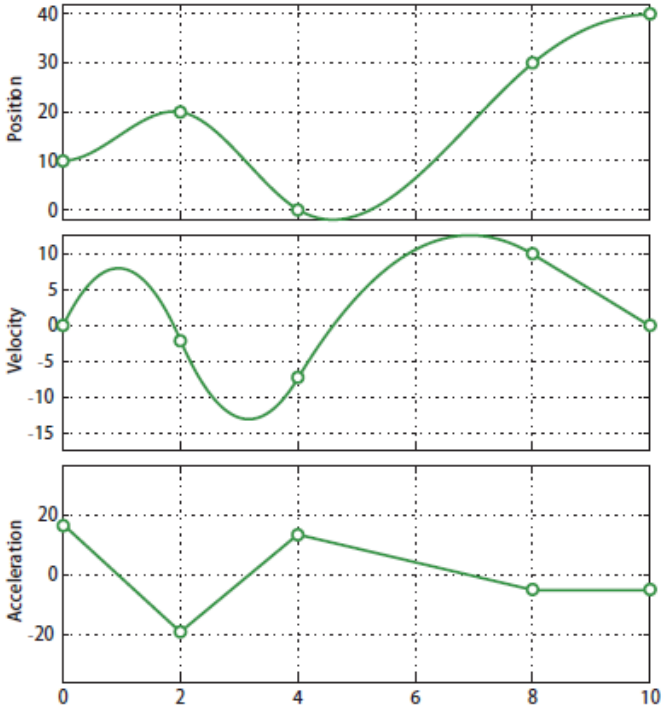
With via points Example:



Via points velocities are arbitrary assigned – large discontinuous acc.



Via points velocities assigned – large discontinuous acc.

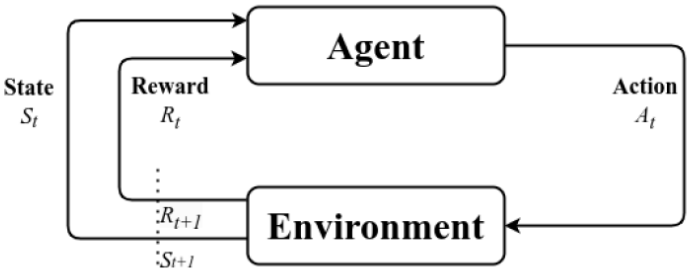


Higher order polynomial – continuous acc.

Defining trajectories with Learning

Training robot policies in simulation

- Training assembly models in Nvidia Factory,
- Testing assembly policies and then moving to the real world,

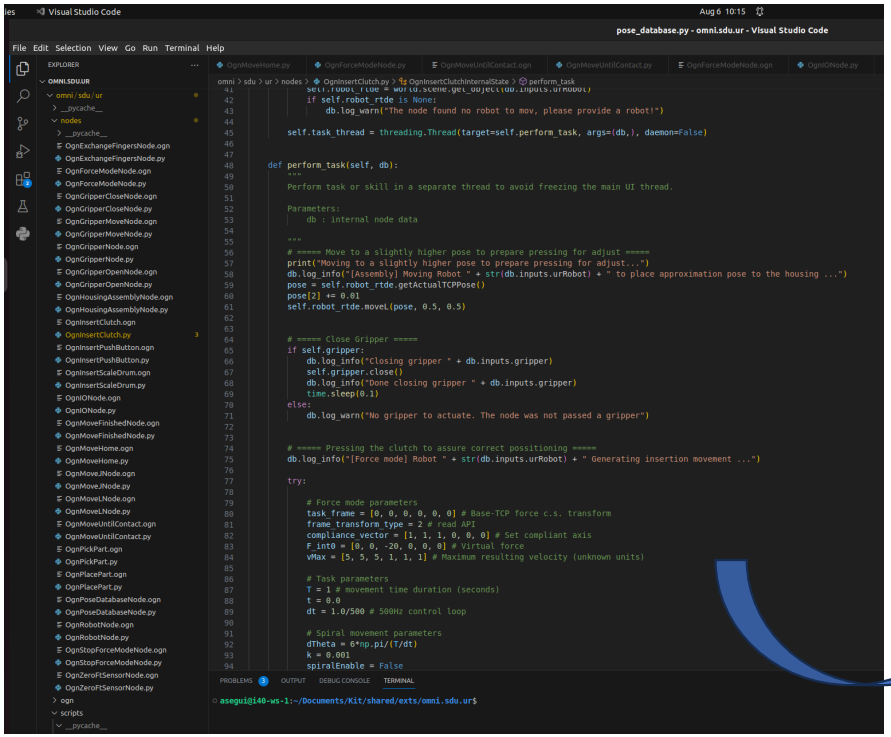


Factory: Fast Contact for Robotic Assembly

Robotics: Science and Systems 2022

From code to "nodes", *by* Adrian Aparisi Segui

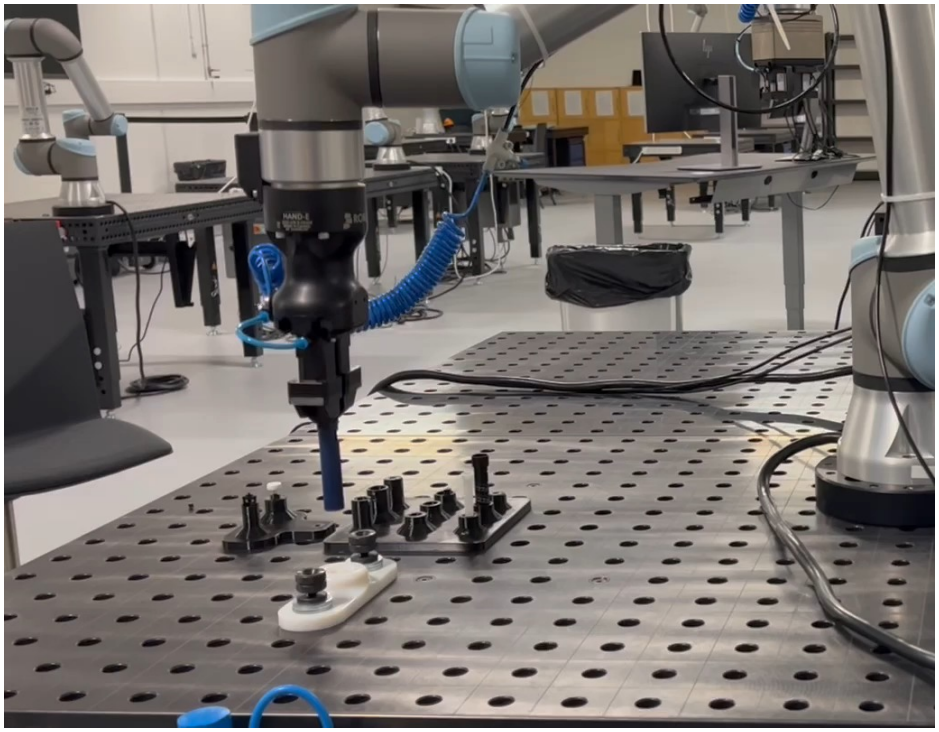
- Ease robot programming by generating blocks that can be easily dragged and dropped by the non-skilled user.

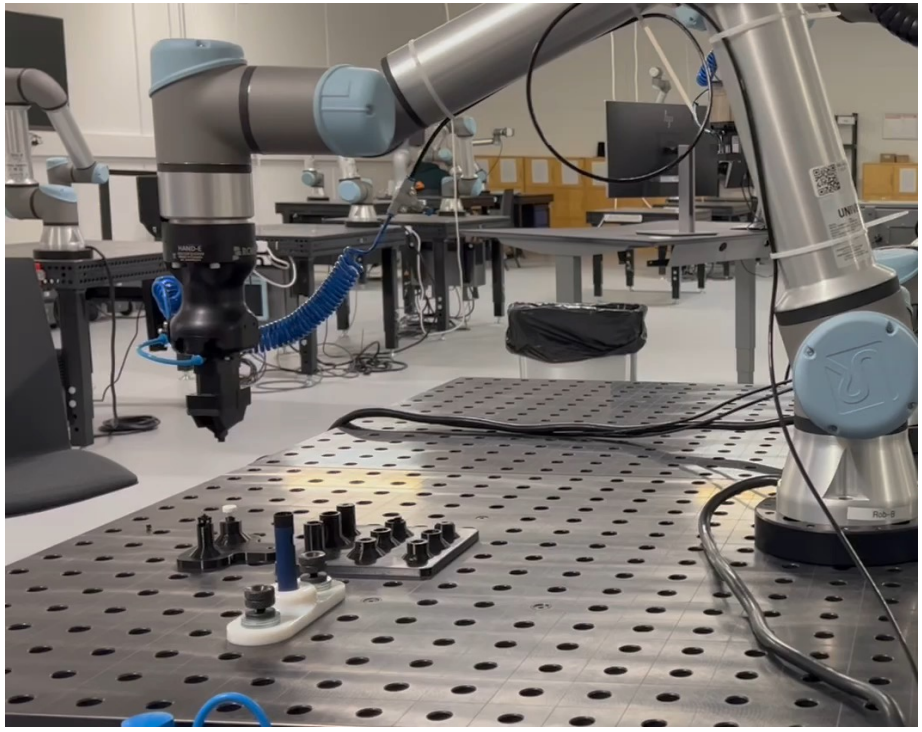


https://syddanskuni-my.sharepoint.com/:v:/g/personal/adapa20_student_sdu_dk/ERZxSpcZhB9Cig4rYPv_zBcBSy6GTPwqp0218HC5wwkt_A?e=lta584

Force mode for assembly

- Some robots such as UR5e have sensors to measure the force applied to the environment, this allows us to program more complex assembly processes

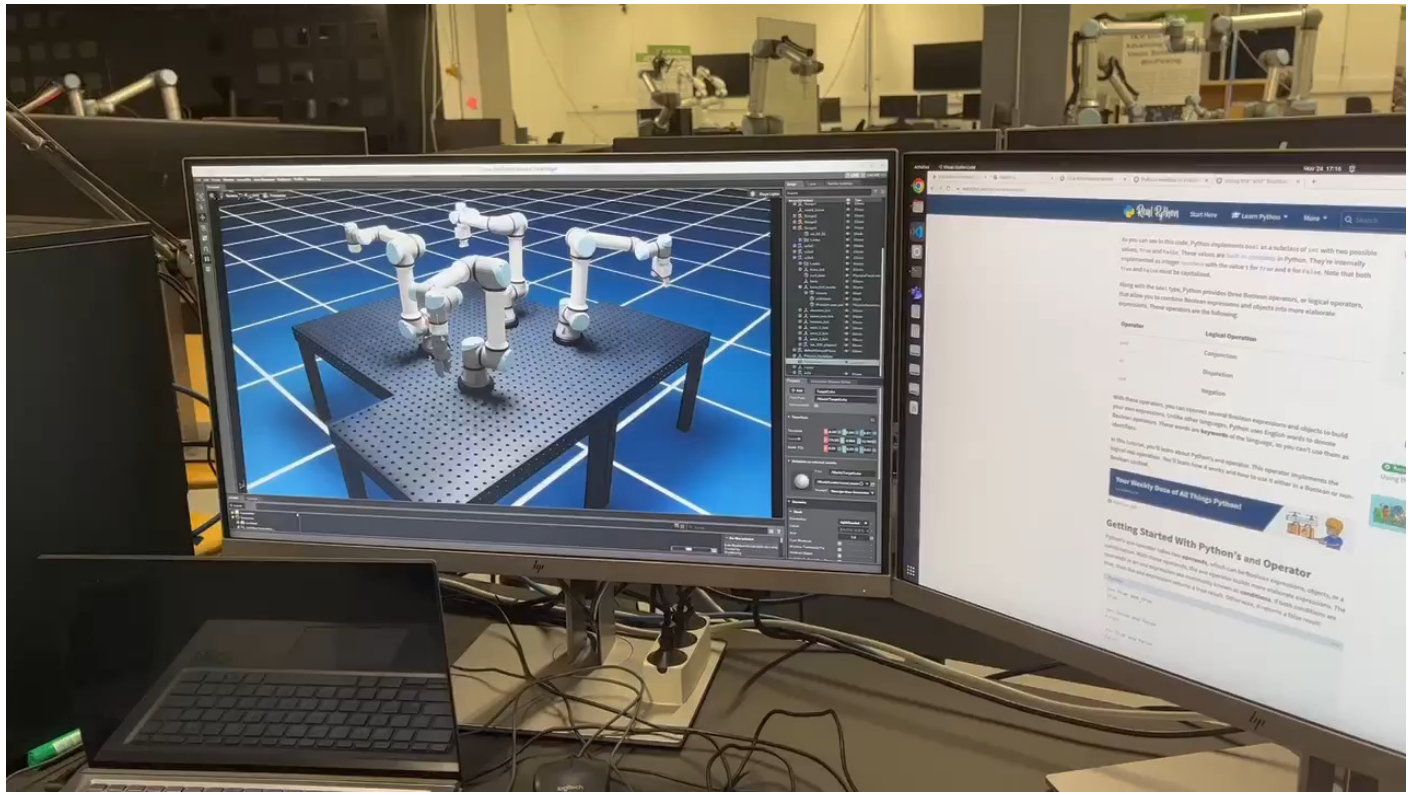




Dynamic simulation of assembly tasks

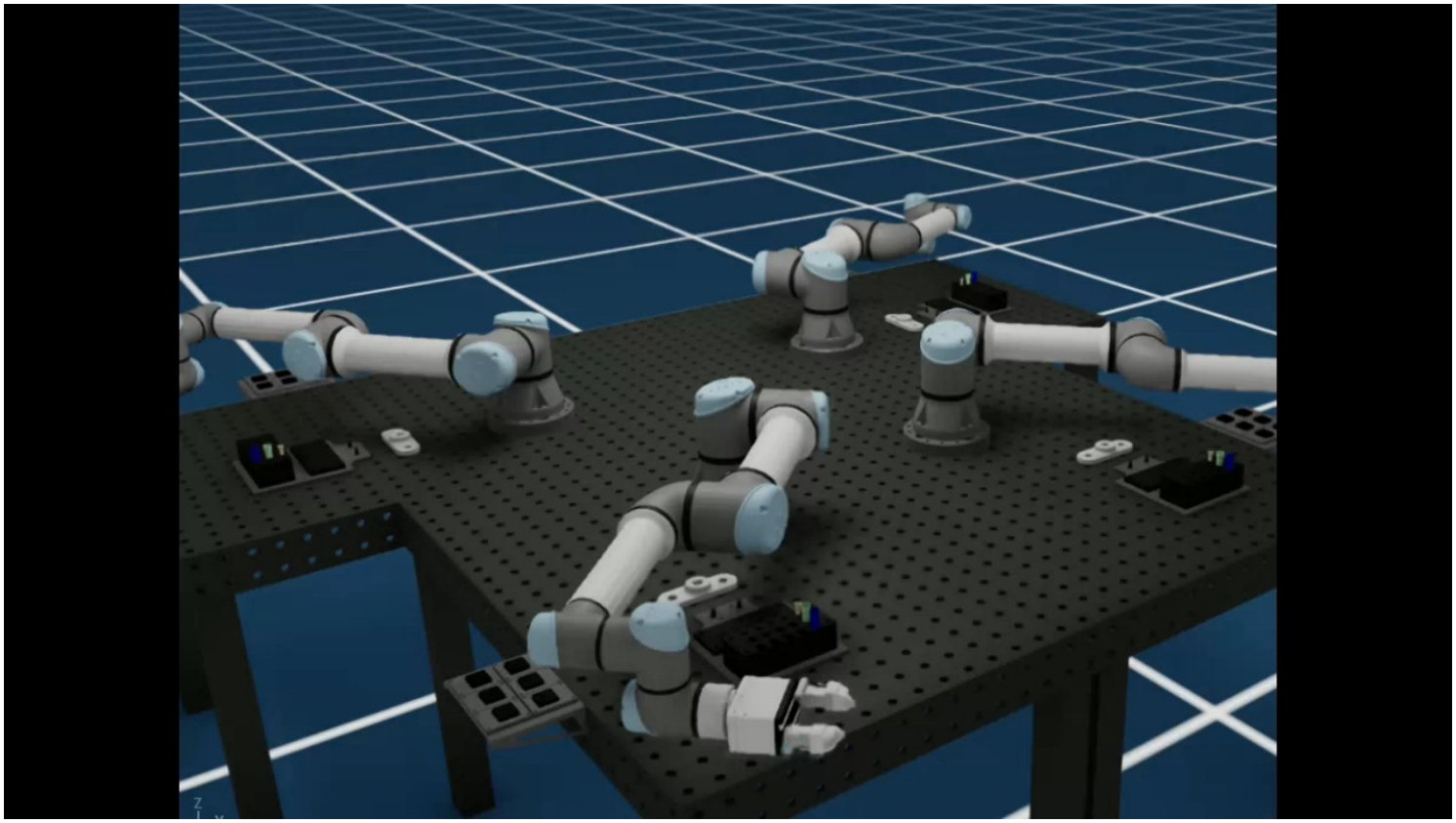
- Task given by Novo Nordisk:
  - Explore the possibilities of simulation
- NVIDIA Isaac Sim
  - Physics simulation
  - FEM
  - Extensions
- Low volume production
- Assembly design/testing

Dynamic simulation of assembly tasks



Dynamic simulation of assembly tasks

Dynamic simulation of assembly tasks



Robotic Grasping

Thanks to:

- J. Li, Grasping lecture, Worcester Polytechnic Institute,
- Berenson, Dmitry, et al. "Grasp planning in complex scenes." *7th IEEE-RAS International Conference on Humanoid Robots*, 2007.

Grasping – is a complex task

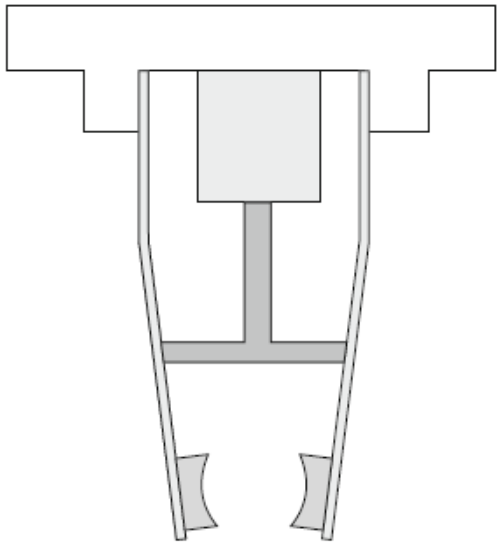
- hand design: high level (number of fingers, kinematic structure, etc.) and low-level (mechanism design, motors, materials, etc.);
- hand control algorithms: high level (find an appropriate posture for a given task) and low-level (execute the desired posture);
- information from sensors (tactile, vision, range sensing, etc.);
- any pre-existing knowledge of objects shape, semantics and tasks (e.g. a cup is likely to be found on a table, should not be held upside-down, etc.);
- all of these add up to a Grasp Planning System...and more!

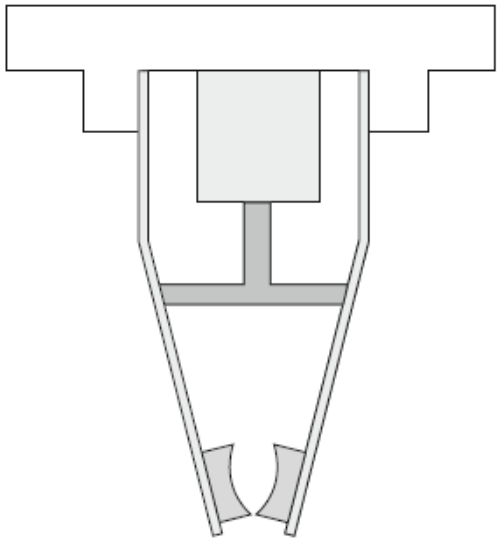
Human vs. robotic grasping

- Human performance provides both a benchmark to compare against, and a working example that we can attempt to learn from. However, it has proven very elusive to replicate:
- the human hand is a very complex piece of equipment, with amazing capabilities;
- humans benefit from an unmatched combination of visual and tactile sensing;
- human continuously practice grasping and manipulation, the amount of data they are exposed to dwarfs anything tried so far in robotics;

Grasping in industrial robotics

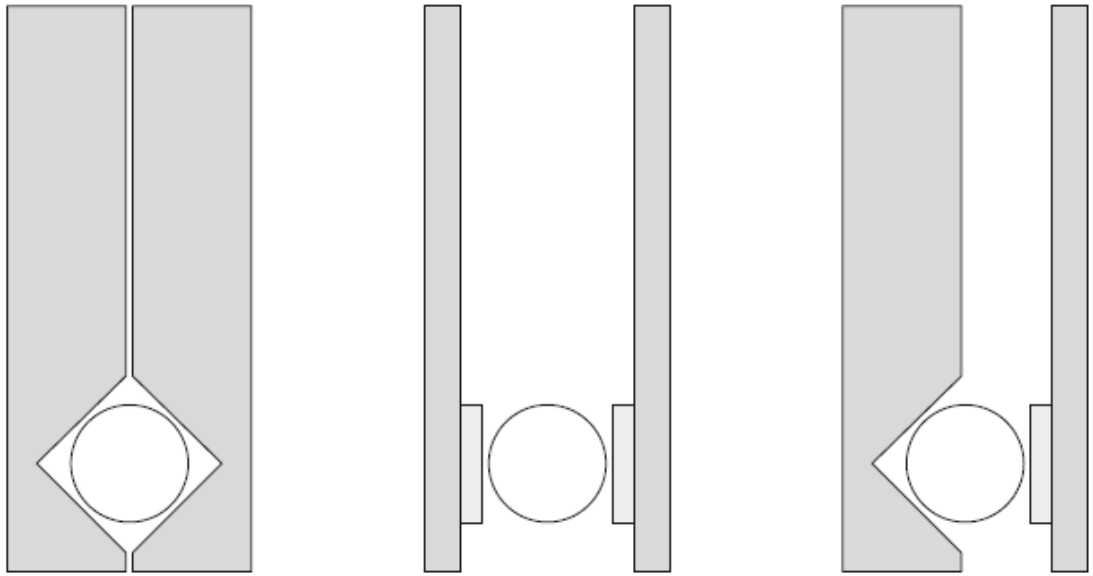
- Grippers with passive spring fingers
- we do not have direct control.





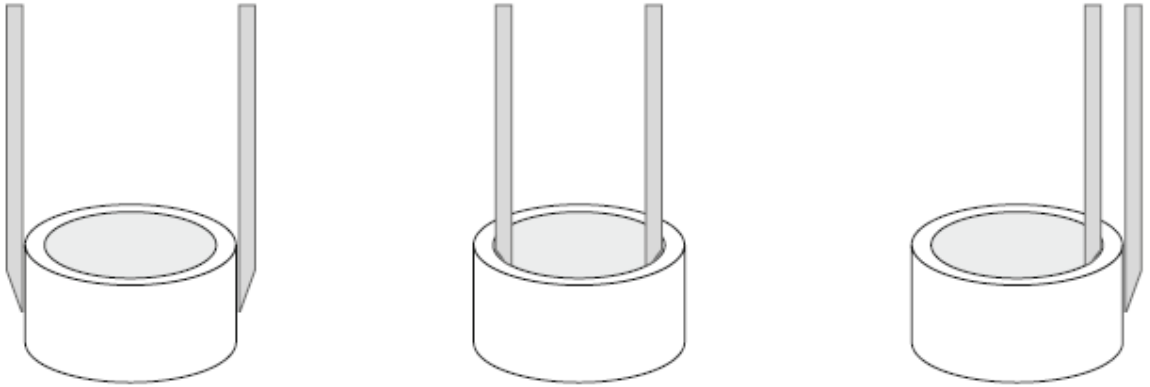
Grasping in industrial robotics

- Form closure, depended on the features of the object and gripper
- Force closure, the movement of the object is restricted based on the applied force, from the gripper.
- combined option



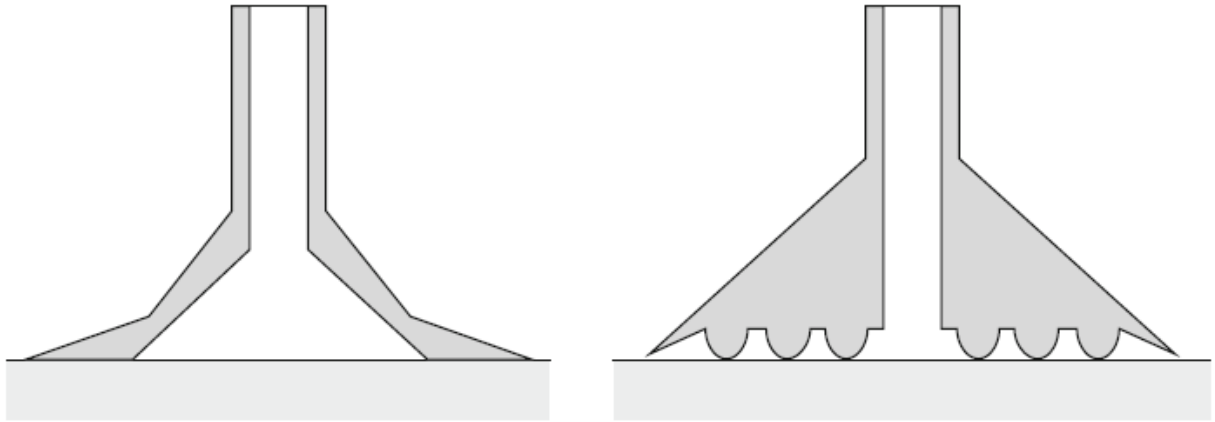
Grasping in industrial robotics

- External grasp
- Internal grasp
- Intermediate grip



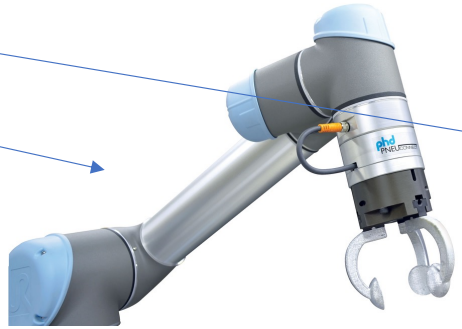
Grasping in industrial robotics

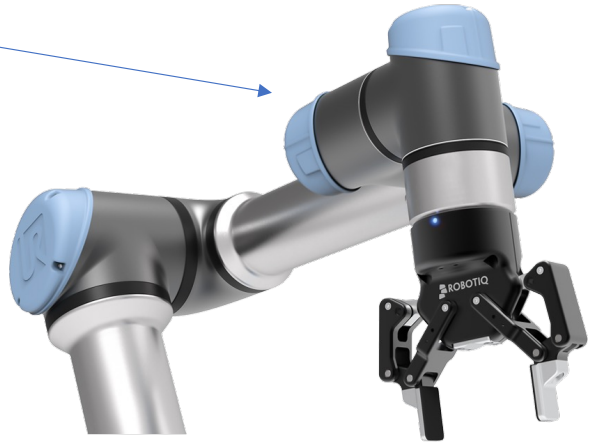
- Vacuum grip, one of the more popular and often implemented methods for object manipulation in industry.



Grippers

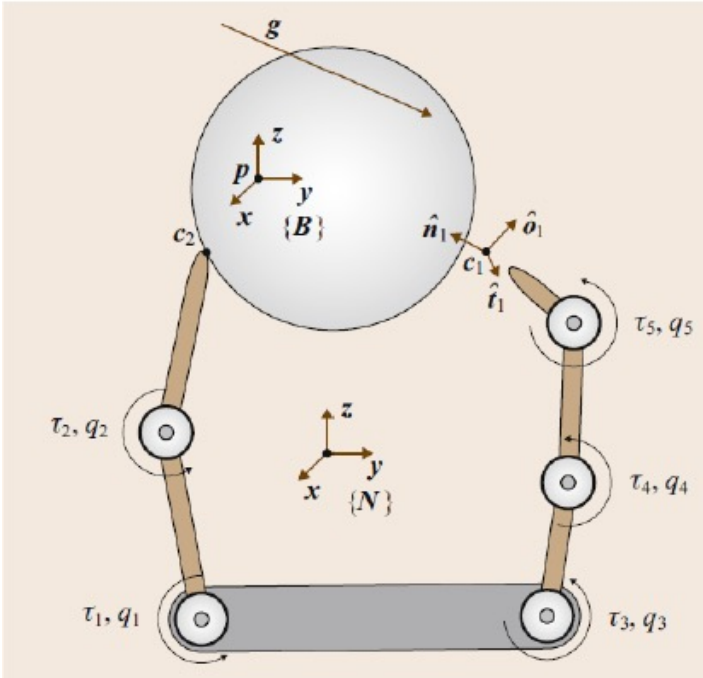
- We distinguish several types of grippers:
  - Servo electric,
  - Pneumatic,
  - Vacuum,
  - Jamming,
  - Etc.





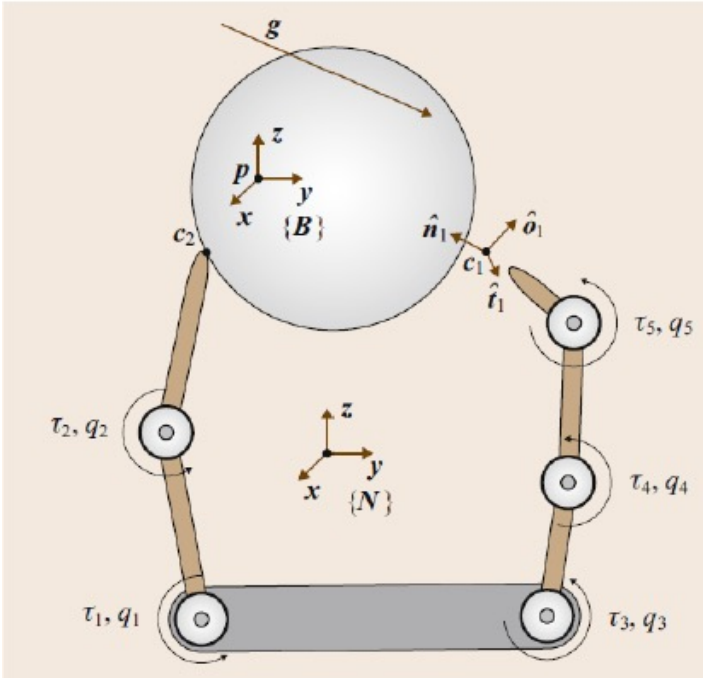
How to define a grasp?

- We have to consider contacts between:
  - Grasping device
  - Object
- The best way to do so is to derive a model.



How to define a grasp?

- With a model we can:
  - Predict the behavior of the hand and object under various loading conditions that may arise during grasping
- Disturbance
  - Inertia force – e.g. fast motion
  - Applied force – e.g. Gravity
- Maintaining a grasp
  - No contact separation
  - No unwanted contact sliding
- Closure grasp
  - The special class of grasps that can be maintained for every possible disturbing load



Model vs real world

A lot of conditions to consider therefore we like to simplify the problem:

- **Real world**: Complex mechanism, soft contacts, soft objects, bounded force.
- **Simplified Problem**: ignore hand mechanism, Assume *n* point contacts, assume rigid object, assume object is fixed

Definition of a point contact

For a one-point contact, we have to consider the following:

- Twist
  - A combination of translational and rotational velocity of the object

$$v_T = [v, \omega]^T$$

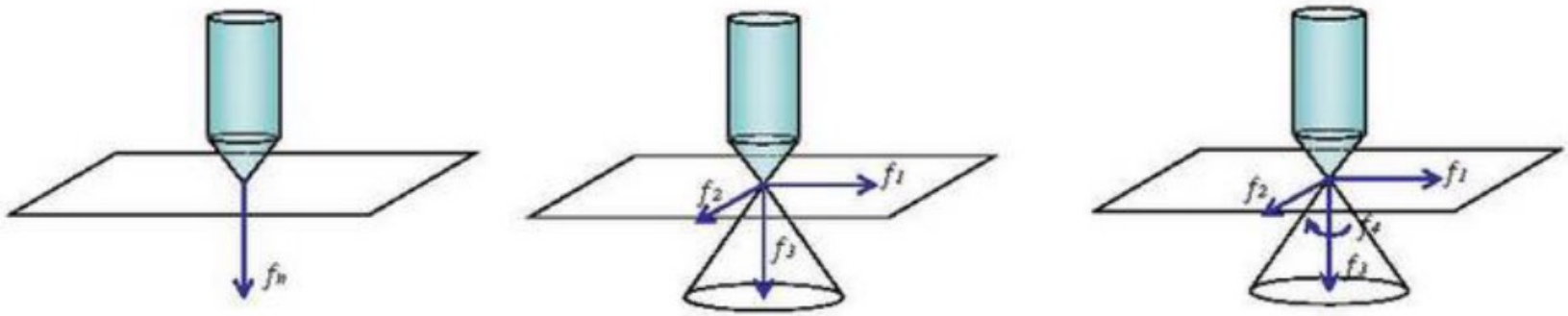
- Wrench
  - A combination of the force and torque applied to the object (at object origin)

$$g = [f, m]^T$$

- Wrench space
  - Space of wrenches applied to the object
    - 3D: 6 dimensional wrench space (3 force, 3 torque)
    - 2D: 3 dimensional wrench space (2 force, 1 torque)

Contacts

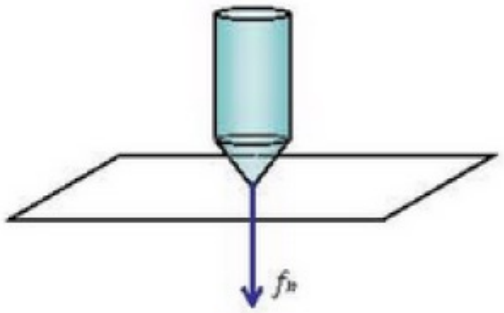
- Point contact without friction
- Hard finger
- Soft finger



Contact modeling

Point contact without friction

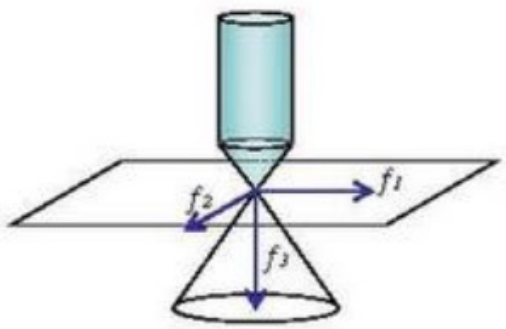
- Contact properties
  - Contact patch is small
  - Contact surface is slippery - no surface friction
- Transmitted to the object
  - Normal component of the translational velocity
  - Normal component of the contact force



Contact modeling

Hard Finger

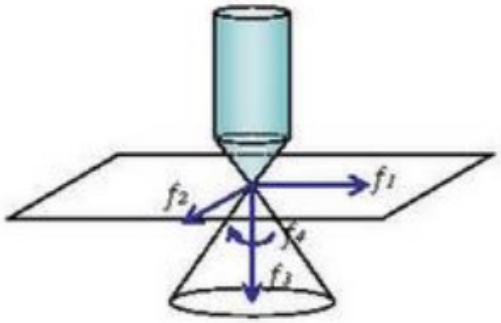
- Contact properties (friction force, but no appreciable friction moment)
  - Small contact patch
  - Large enough surface friction
- Transmitted to the object
  - All three components of the translational velocity
  - All three components of the contact force
  - No angular velocity or moment



Contact modeling

Soft Finger (SF)

- Contact properties (appreciable friction moment)
  - Large enough contact patch
  - Large enough surface friction
- Transmitted to the object
  - All three components of the translational velocity
  - All three components of the contact force
  - Normal component of contact moment



Contact modeling

Friction cone

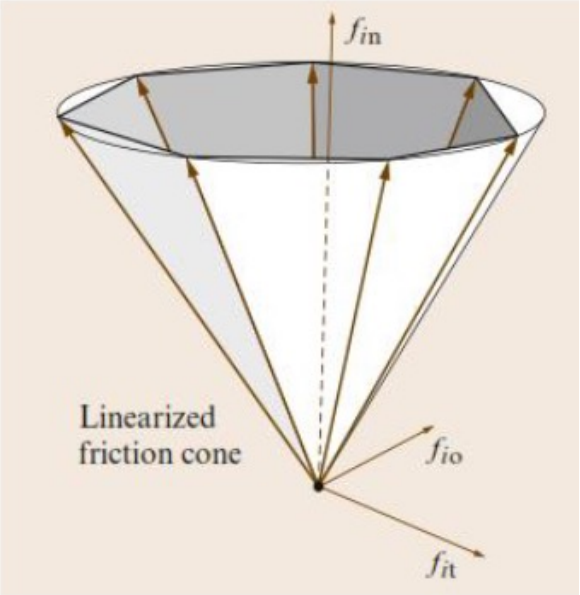
- The set of forces that can be applied at a contact point without sliding on the object
- Friction cone for *i*th contact point is the set

$$\mathcal{F}_i = \{(f_{in}, f_{it}, f_{io}) | \sqrt{f_{it}^2 + f_{io}^2} \leq \mu_i f_{in} \}$$

- f_{in} is the force applied normal to the surface
- f_{io} and f_{it} are the forces applied along the surface

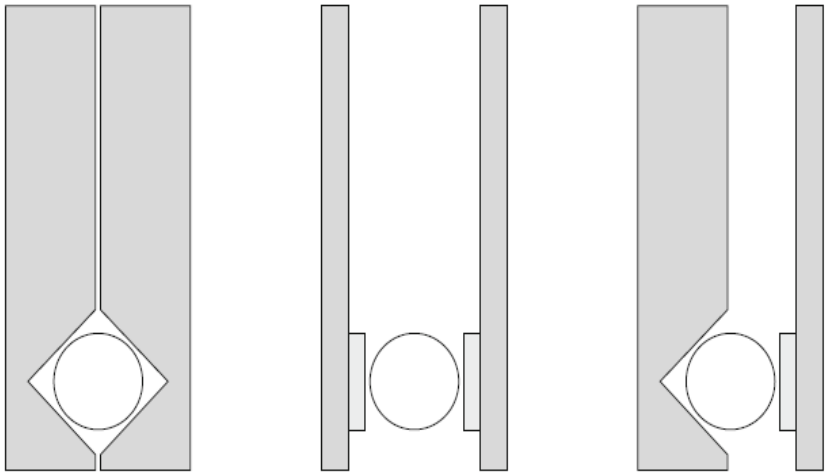
Note:

- Coulomb friction
- Depends on coefficient of friction between hand and object
- Bigger μ implies wider friction cone



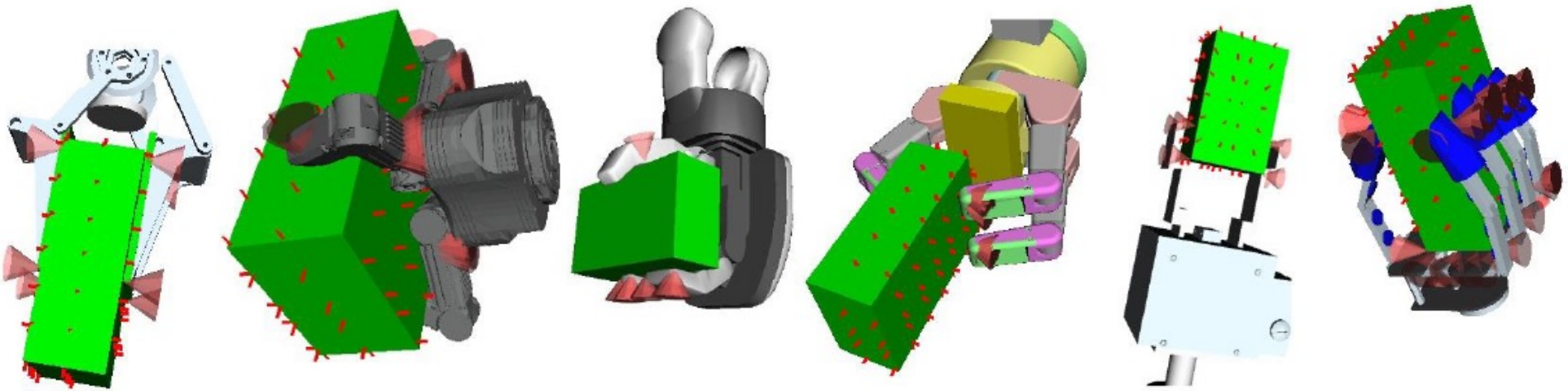
Grasping in industrial robotics

- Form closure, depended on the features of the object and gripper
  - You need at least N+1 contacts to achieve first-order form closure, where N is the number of DOF of the object
- Force closure, the movement of the object is restricted based on the applied force and friction material properties.
- combined option



Grasp planning

- Grasp planning can be calculated based on a set of precomputed set of stable grasps



Grasp planning

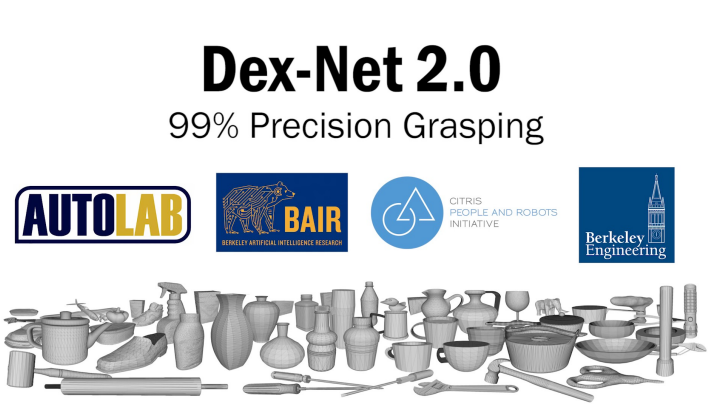
This concept is not new,

- Extensive grasping databases exits: http://grasping.cs.columbia.edu/
- Reuse the 3D models from the Princeton Shape Benchmark (PSB)
  - Well known academic dataset of 1,814 models
  - All models resized to “graspable” sizes
  - PSB models were not originally selected with an eye towards robotic grasping
    - Some of the models are not obvious choices for grasping experiments.
- Provide grasps at 4 scales
  - …because grasping is scale dependent
  - .75, 1.0, 1.25 and 1.5 times the size of each model
  - 7,256 3D models in all

Other approaches

- Using learning to find the appropriate grasp for a defined object
  - GraspNet-1Billion https://graspnet.net/tasks.html
  - DexNet https://berkeleyautomation.github.io/dex-net/#dexnet_21

https://graspnet.net/images/120_pred.gif



https://www.youtube.com/watch?v=i6K3GI2_EgU

https://www.youtube.com/watch?v=GBiAxoWBNho&t=17s

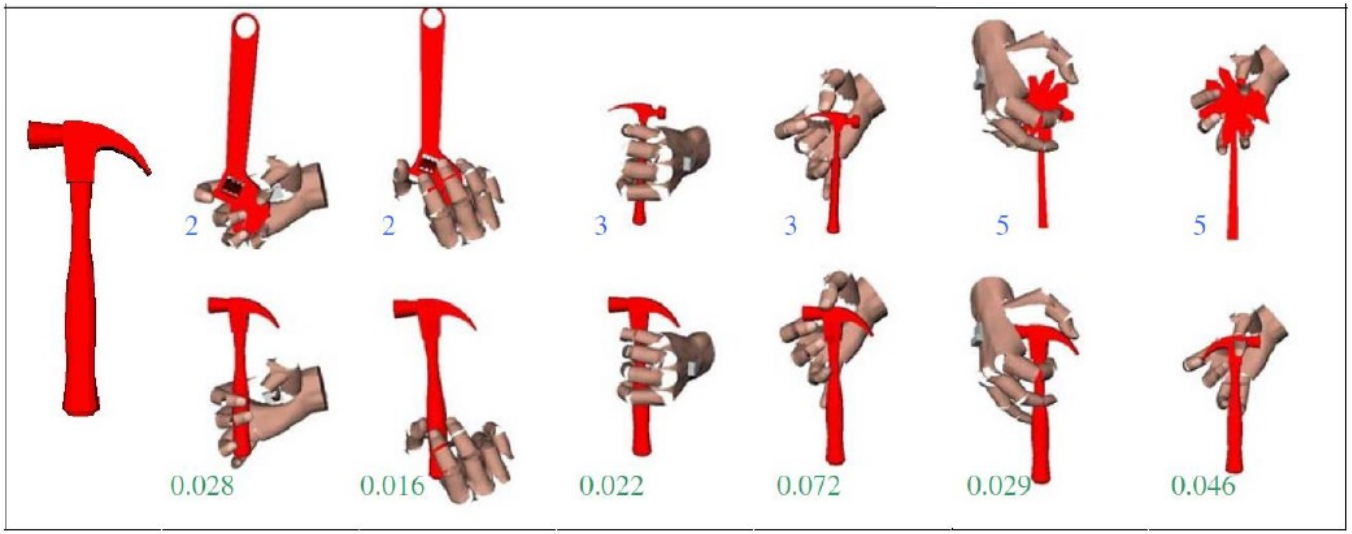
Grasp planning

A grasp based on the database is computed as:

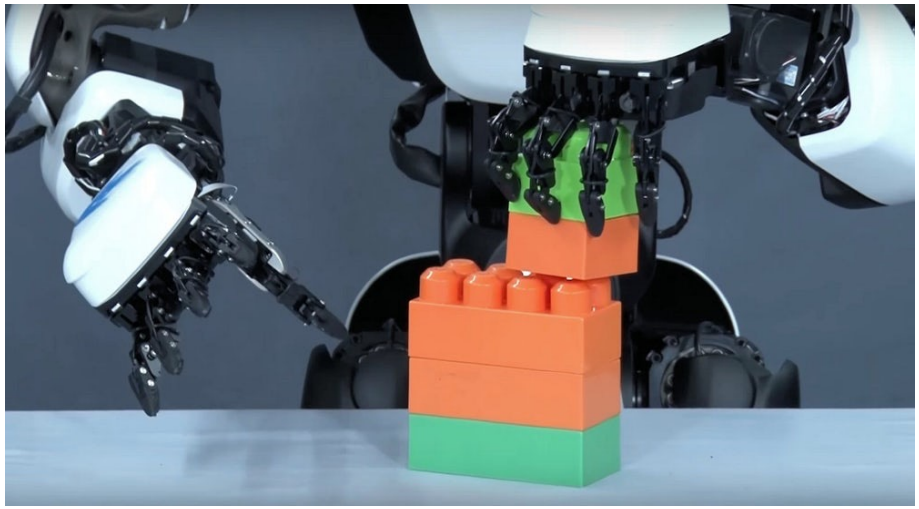
- Shape matching, collocating and grasp computing

Performance

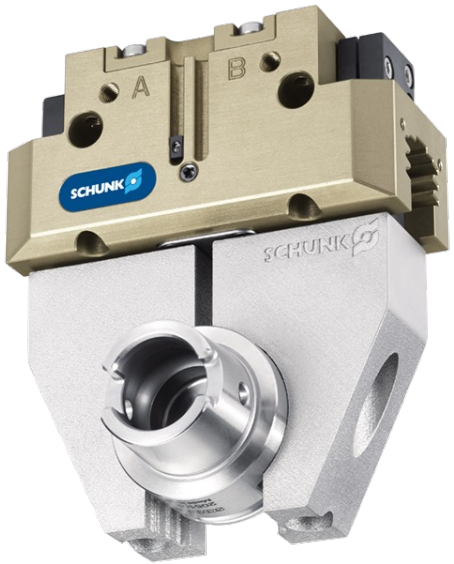
- 20 seconds, from shape matching to final output



Industrial grasping



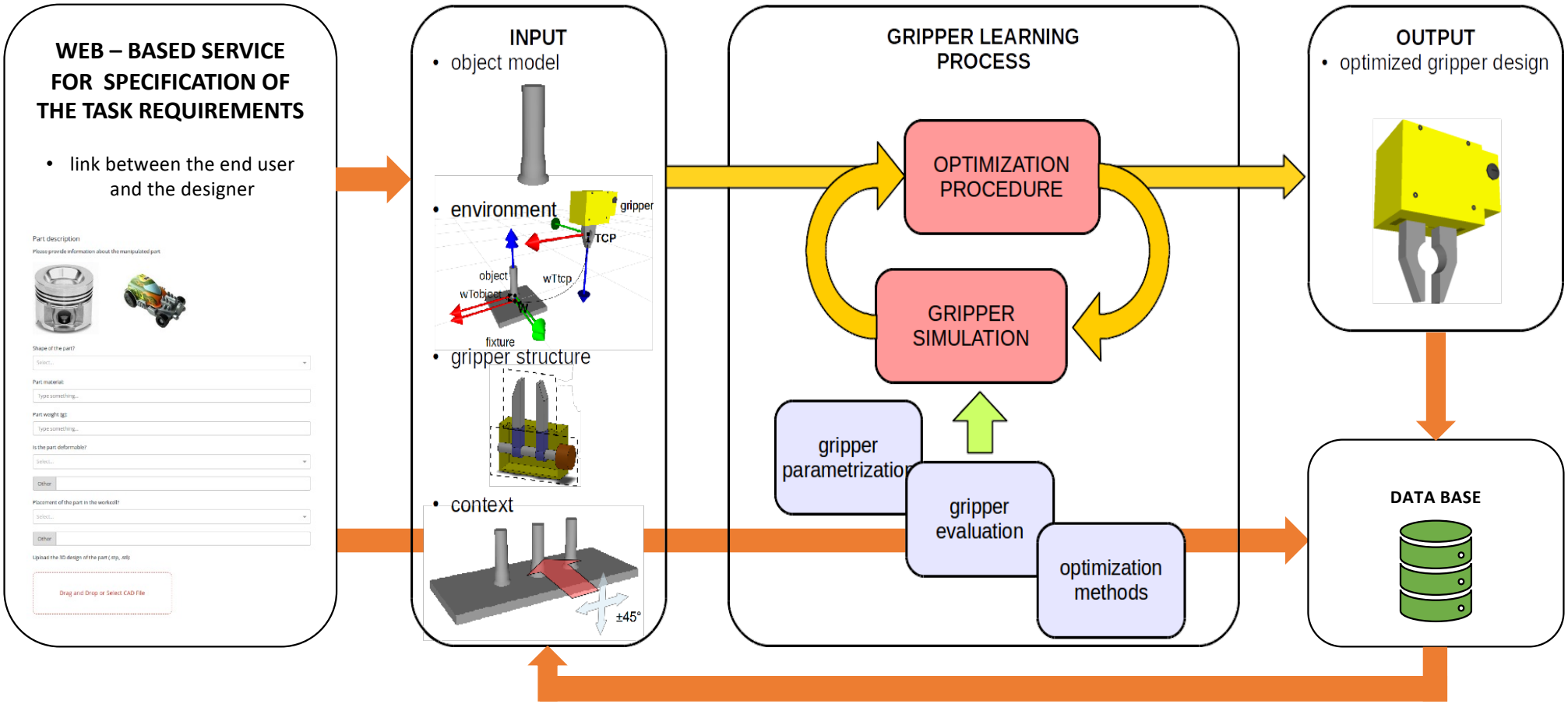
Dexterous grippers for industrial assembly



Task specific grippers for industrial assembly

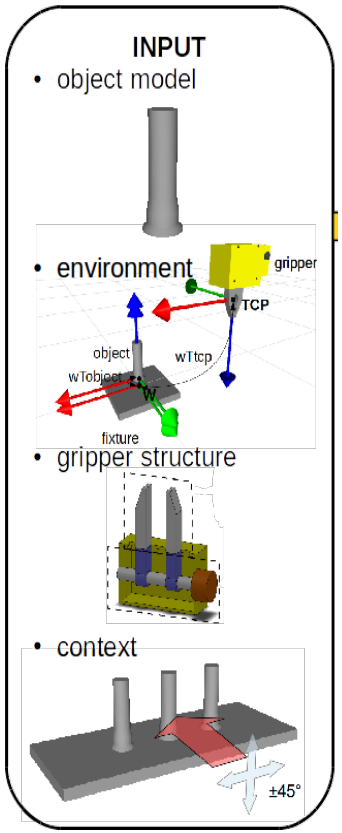
https://www.therobotreport.com/wp-content/uploads/2019/01/toyota-thr3-robot-6.jpg

https://schunk.com/fileadmin/user_upload/Live_ab_2016/06_Service/Tools_Downloads/eGrip/eGRIP_Metall_PGN_80_TRIBOS_geschlossen.png



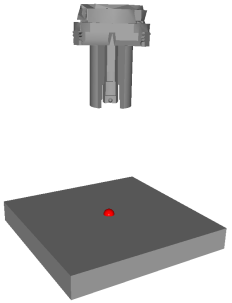
Aljaz Kramberger – alk@mmmi.sdu.dk, SDU Robotics, The Maersk Mc-Kinney Moller Institute.

Design process setup based on the gathered information

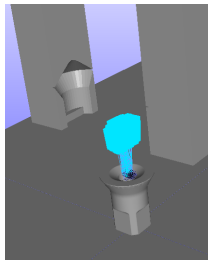


INPUT:
- part model
- task context
- nominal grasp

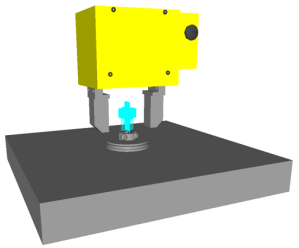
create Robwork workcell



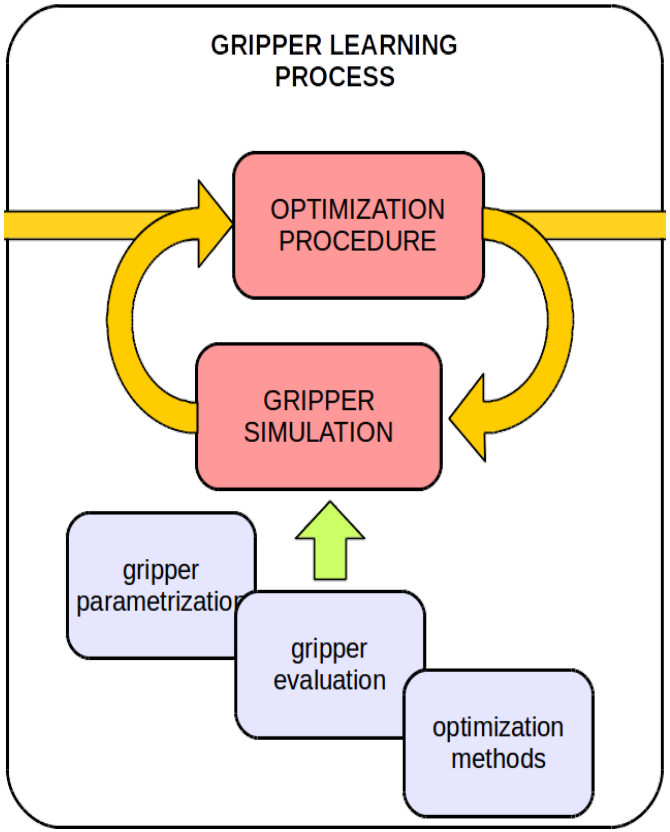
generate grasp set



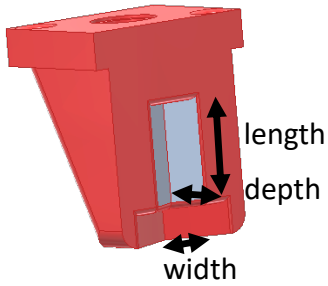
define gripper parametrization



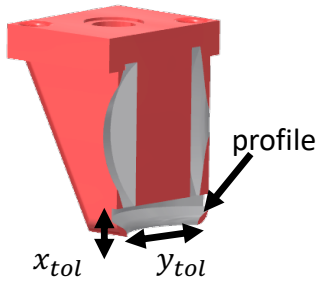
SDU Aljaz Kramberger – alk@mmmi.sdu.dk, SDU Robotics, The Maersk Mc-Kinney Moller Institute.



Parameterized design

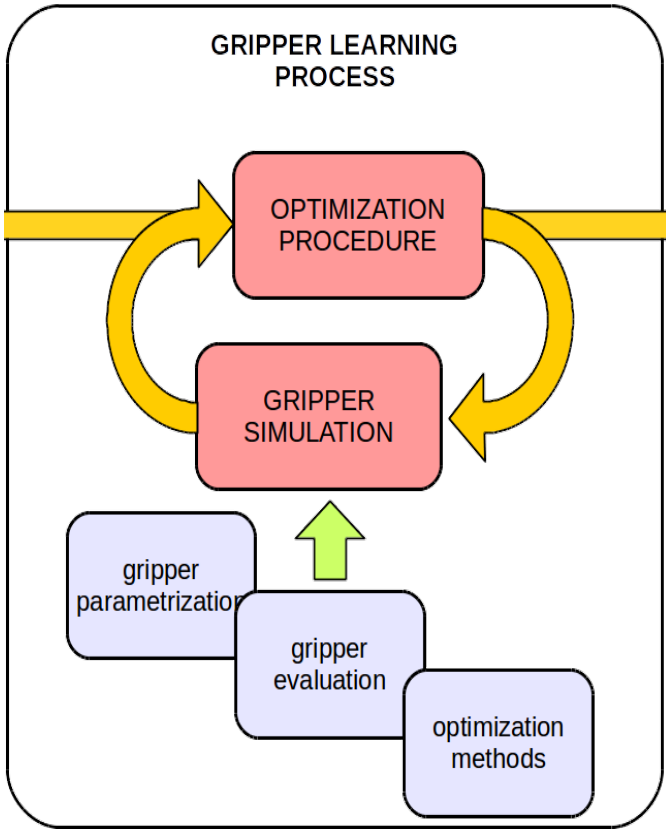


Imprint design

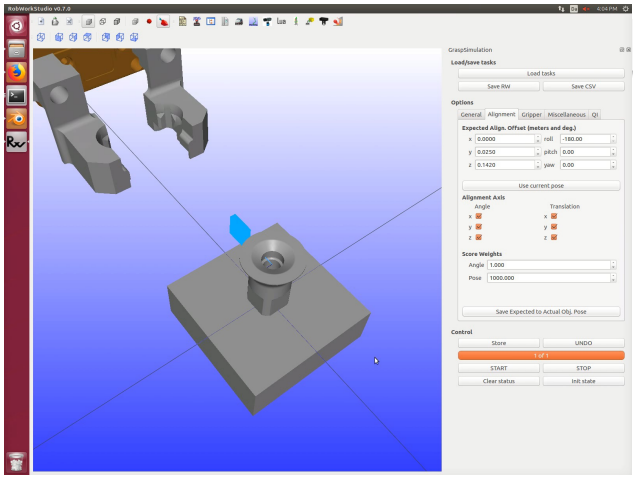


- **Parameterized design:**
  - Geometrical parameters, are used to define the cut-outs of the base finger shape.
  - Limited to simple parameters which can be simply measured.
- **Imprint design:**
  - The method is based on extracting the geometrical features of the manipulated object and transferring them to the base finger shape.
  - Smoothening parameters are used for shape alteration.

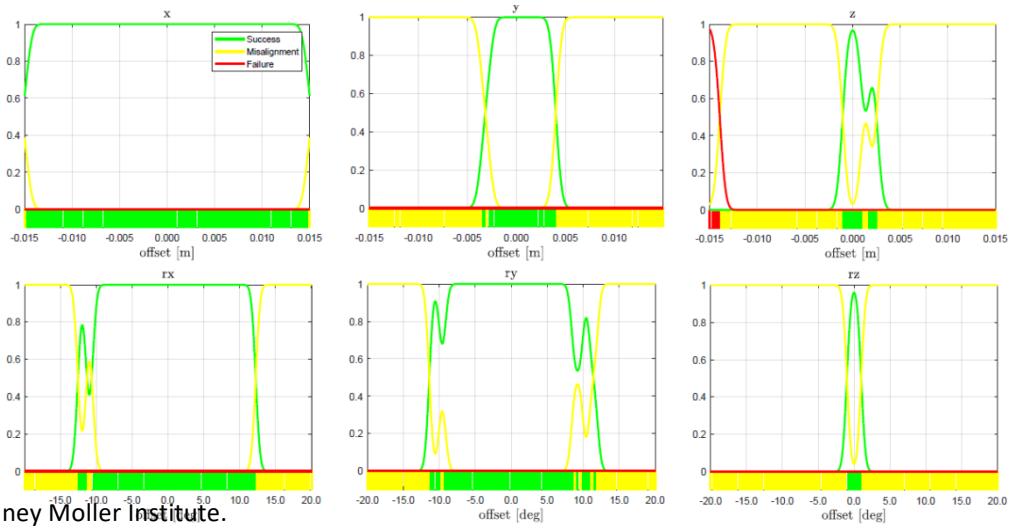
SDU Aljaz Kramberger – alk@mmmi.sdu.dk, SDU Robotics, The Maersk Mc-Kinney Moller Institute.



- **Success Index** *S* – the percentage of successfully executed grasps
- **Robustness Index** *R* – gripper performance in the presence of the process noise
- **Alignment Index** *A* – the gripper ability to force predictable poses on the grasped object
- **Coverage Index** *C* – the gripper versatility in approaching the object
- **Wrench Index** *W* – how secure the grasps executed by the gripper are



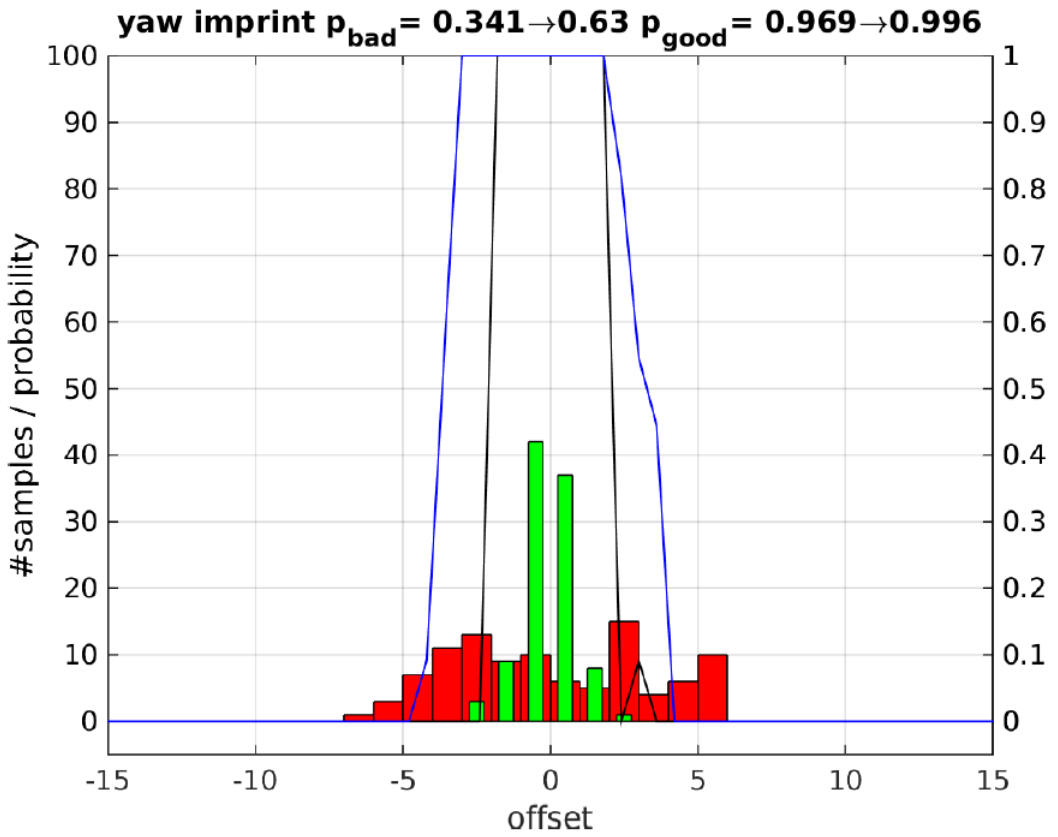
Dynamic testing results



SDU Aljaz Kramberger – alk@mmmi.sdu.dk, SDU Robotics, The Maersk Mc-Kinney Moller Institute.

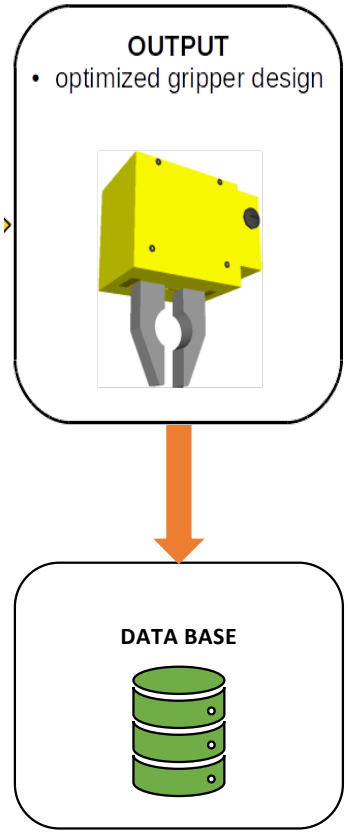
Optimization

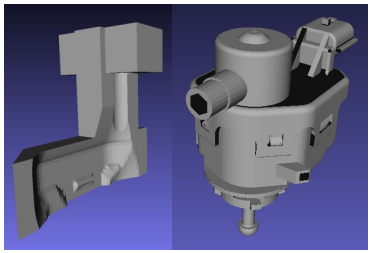
- Depending on the design and simulation outcomes, the fingers shape may not be optimal for the specific use-case.
- To achieve optimality, we apply an numeric optimization process, to refine the initial design.
- The process changes and simulate the design parameters in an iterative manner, until the desired performance is achieved.

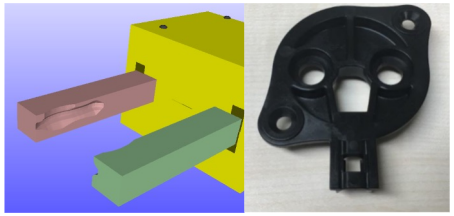


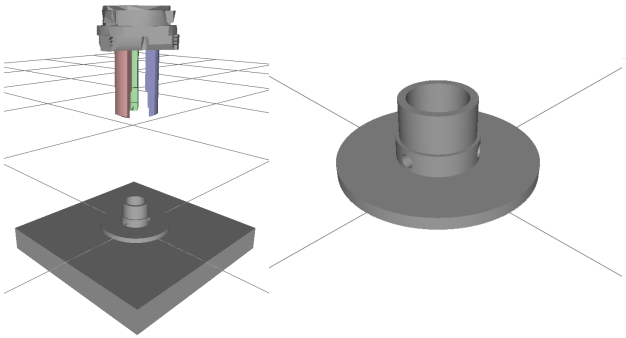
SDU Aljaz Kramberger – alk@mmmi.sdu.dk, SDU Robotics, The Maersk Mc-Kinney Moller Institute.

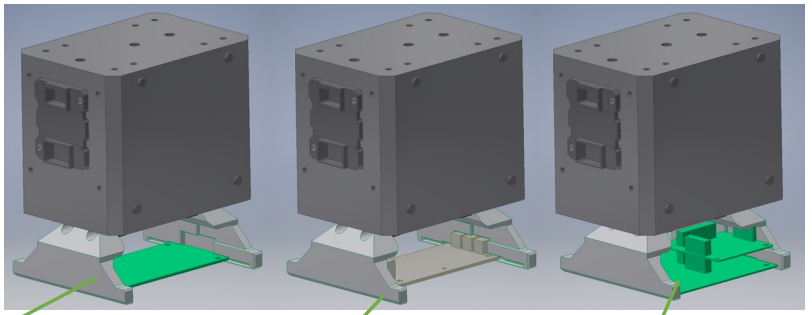
Some design examples

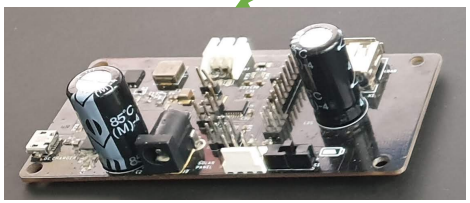


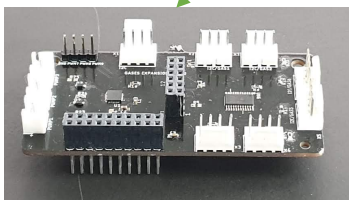


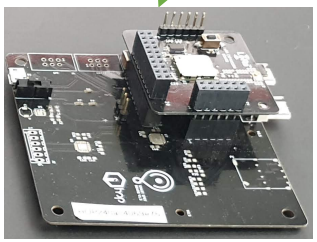












SDU Aljaz Kramberger – alk@mmmi.sdu.dk, SDU Robotics, The Maersk Mc-Kinney Moller Institute.

Thank you for your attention